

Anastasi

Smart Objects & LLN

Smart objects

Dispositivo generico con hardware per **comunicare** (WiFi o BLE), **computare** (microcontrollore con CPU, RAM, ecc.) effettuare **sensing** (sensori, attuatori e un ADC per digitalizzare le grandezze rilevate) e un modulo per garantire erogazione di energia (batteria o spina della corrente).

I sensori invece si dividono in:

- **Passivi:** richiede una fonte esterna per essere interrogato, come un tag RFID. Non ha una fonte di alimentazione interna.
- **Attivi:** Trasmettono un segnale, ha una fonte di alimentazione interna
- **Semi-passivi:** richiede una fonte esterna ma ha una fonte di alimentazione interna.

Un esempio di smart object è il Dongle.

Tipi di comunicazione nelle LLN.

- **One-to-One:** La sorgente invia al vicino, quest'ultimo invia al suo vicino e così via fino alla destinazione.
- **Many-to-One:** Molte sorgenti e una destinazione (sink), tipico delle LLN in cui i sensori inviano i dati ad una destinazione comune passando tutti per il BR.
- **One-to-Many:** Una sorgente e molte destinazioni. Meno frequente rispetto alle precedenti, però in alcuni casi è utile per inviare comandi ai sensori per cambiare le loro configurazioni o nelle reti di attuatori

Routing nelle LLN.

Flat Multi-hop

Per le **WSN dense** (piccola area, molti nodi). Ogni nodo è uguale agli altri: invia il messaggio al vicino, e così via fino a destinazione (in genere il sink o il BR). Percorsi lunghi:

- **Grande delay.**
- **Effetto funneling:** BR diventa bottleneck → consuma più energia → si scarica → single point of failure, la rete smette di funzionare.
- Tanti link, più probabile la perdita di pacchetti (ritrasmissioni → energia).

Hierarchical Multi-hop

Ogni nodo invia i messaggi al proprio super node, i super node inoltrano solo ad altri super node, fino al sink, connesso a più super node insieme. Percorsi più brevi:

- Se un super node si guasta → si deve riconfigurare la rete.
- Scalabilità più costosa, perché i super node costano molto di più.

Mobile data collectors (Mobility-based)

Per le **WSN sparse** (vasta area, pochi nodi). Un nodo mobile periodicamente si avvicina ai sensori e raccoglie i dati con una comunicazione one-to-one ad un solo hop.

- Il nodo mobile è costoso e il movimento può essere difficoltoso.
- Comunicazioni limitate dal movimento.
- La velocità del nodo mobile deve essere coerente con il tempo necessario alla trasmissione.
- Collector e sensori devono potersi sincronizzare.

Energy Management

Perché è necessario limitare il consumo energetico nelle LLN?

I dispositivi sono limitati: hanno **poche risorse energetiche** e **non è facile ricaricarli**:

- **Low Power Devices:** Usare dispositivi che consumano poco.
- **Energy Harvesting:** Usare dispositivi con un sistema di raccolta di energia.
- **Energy Conservation.**
- **Energy Efficient Protocols & Applications.**
- **Cross-Layering:** Permettere alle applicazioni di interagire con i livelli sottostanti dello stack protocollare per cambiare le configurazioni in base a ciò che si deve fare.

Energy harvesting in generale

Tecniche per raccogliere energia dall'ambiente esterno e stoccarla nel dispositivo (comunque necessaria, perché l'energia esterna non sempre è disponibile). Ci sono vari modi per raccogliere energia dall'esterno:

- **Energia umana:** termina, cinetica.
- **Energia meccanica:** pressione, vibrazioni.
- **Ambientale:** solare o frequenze radio.

Energy management in generale.

Tecniche per risparmiare energia. La componente che consuma di più è la **radio** (81%) , dobbiamo utilizzare la radio il meno possibile. La radio può stare in uno di questi tre stati:

- **Active:** sta trasmettendo/ricevendo.
- **Sleep:** spenta, non può trasmettere/ricevere.
- **Idle:** accesa ma non trasmette/riceve.

Tre insiemi di tecniche utilizzabili contemporaneamente:

- **Duty-cycling.**
- **Data-driven.**
- **Mobility-based:** Utilizziamo un dispositivo mobile per ricavare i dati: no funneling effect e migliora il tempo di vita del dispositivo.

Energy management - Data driven approach.

Agiamo sui dati da trasmettere per tenere la radio attiva per meno tempo.

Data reduction: Riduco i dati da trasmettere → trasmetto per meno tempo

- **Data aggregation:** Unire più pacchetti per trasmettere meno byte.
- **Data prediction:** Derivare un modello del fenomeno, poi condividerlo (invia i parametri) alla destinazione. La sorgente può evitare di inviare tutti i campioni, quelli mancanti verranno dedotti usando il modello. Il modello periodicamente deve essere aggiornato e poi condiviso di nuovo.
- **Data compression:** Codificare le informazioni usando un numero minore di bit rispetto alla rappresentazione originale.
 - **Lossless compression:** riduce senza perdere informazioni, performance limitate.
 - **Lossy compression:** performance maggiori, ma è possibile perdere dati.
- **Energy-efficient sampling:** meno dati prodotti → meno dati da trasmettere → trasmetto per meno tempo.

Gli approcci data-driven **non sono sufficienti** a raggiungere una buona riduzione.

Energy management - Mobility base approach.

Non lo ha spiegato nel dettaglio.

Differenza tra Data Aggregation e Data Compression

Data aggregation → Agire su molteplici pacchetti per ridurre la quantità di dati finali inviati al sink.

Data compression → Agire su un singolo pacchetto per ridurre la dimensione del payload.

Energy management - Data Driven – Data aggregation

- **MAC Layer Data aggregation:** Accorpare più pacchetti (intestazione comune) in uno solo, trasmetto l'intestazione solo una volta.
- **Cluster-based:** Divido i nodi in cluster (n nodi e un cluster head) i nodi inviano i dati al CH, li aggrega e poi invia al sink.
- **Tree-based:** Ogni nodo invia la misurazione al padre. Ogni padre aggrega (sfruttando anche conoscenze riguardo l'applicazione) i messaggi dei figli e poi invia un solo pacchetto al proprio padre.

L'efficacia dell'aggregazione è misurata dall' **aggregation factor**, nel caso della MAC Layer Data aggregation:

$$\alpha = \frac{D_{agg}}{D_{no-agg}} = \frac{H+N*P}{N(H+P)}, \text{ con } N \text{ il numero di pacchetti, } H \text{ il peso dell'intestazione e } P \text{ del payload.}$$

Duty cycling e ASCENT (algoritmo e analisi delle performance)

Riduciamo la quantità di tempo in cui la radio è active:

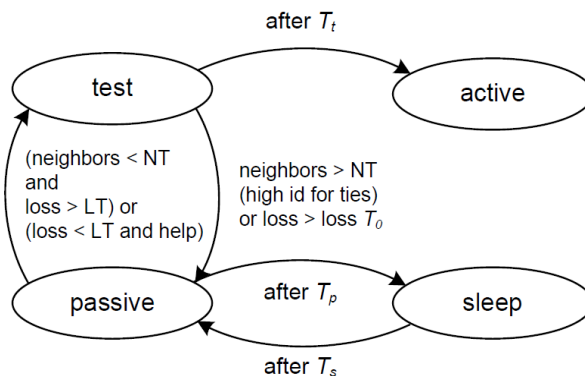
- **Power management:** Sfruttare i periodi di idle nella comunicazione per mettere la radio in sleep.
- **Topology control:** Sfruttare la ridondanza di nodi della rete per mantenere il massimo numero di nodi in sleep senza perdere il funzionamento del sistema. I nodi attivi si alterneranno con gli altri secondo una certa politica.
 - **GAF (Geographic Adaptive Fidelity):** Ogni nodo conosce la sua posizione (GPS). Lo spazio è diviso in quadrati e in ogni quadrato viene scelto solo un nodo da tenere attivo (elezione fatta dai nodi, grazie al GPS sanno in quale quadrato si trovano). I quadrati sono dimensionati in modo che due nodi in due quadrati adiacenti possano sempre comunicare (dimensionato in base al raggio di trasmissione).
 - **ASCENT.**

ASCENT

Considerato per il random deployment, un nodo decide se rimanere attivo basandosi su misurazioni locali fatte dal nodo stesso. Un nodo può essere in uno di questi stati:

- **Active:** Inoltra ma non raccoglie misure di rete.
- **Sleeping:** Non inoltra e non raccoglie misure di rete. (radio spenta).
- **Test:** Inoltra e raccoglie misure della rete.
- **Passive:** Non inoltra ma raccoglie misure della rete (radio attiva).

Raccogliere misure di rete → Riceve pacchetti degli altri.



1. Inizialmente il nodo è in **sleep**.
2. Dopo T_s → **passive**.

Il nodo può ricevere **HELP** (broadcast) dal sink e messaggi da vicini in cui indicano il loro stato. Supponiamo che il sink comunichi la perdita di pacchetti.

3. Il nodo in **passive** monitora:
 - a. Vicini attivi < NT **AND** perdita > LT → **test**.
 - b. **HELP AND** perdita < LT → **test**.
 - c. Dopo T_p → **sleep**.

In **test** il nodo ha la radio attiva e deve decretare se è necessario.

4. Il nodo in **test** monitora:
 - d. Vicini attivi > NT → **passive** (va in passivo il nodo con ID minore).
 - e. Perdita > LT → **passive**. (troppi attivi, tante collisioni)
 - f. Dopo T_t → **active**.

Performance di ASCENT

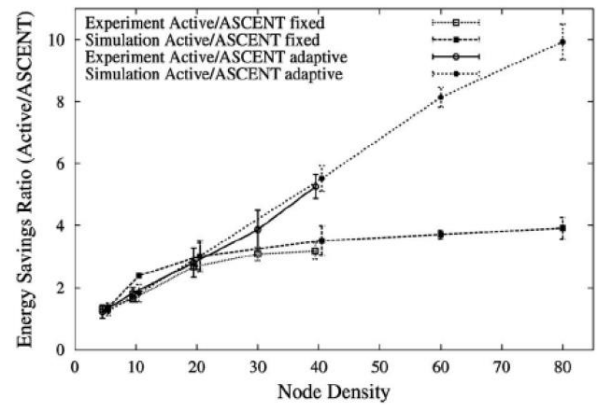
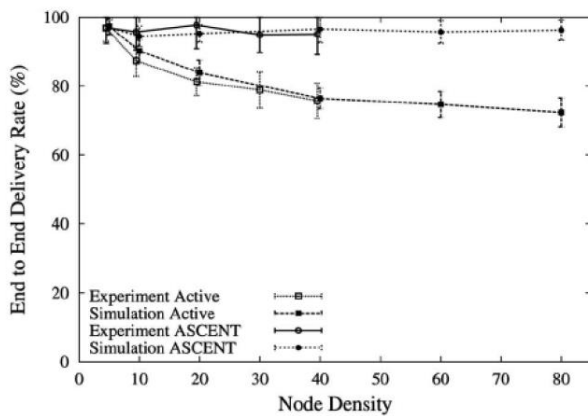
Analizziamo due metriche:

- **Delivery ratio DR** → Percentuale di pacchetti inviati correttamente al sink.
- **Energy Saving Ratio ESR** → Rapporto tra energia consumata quando tutti i nodi sono attivi (NO ASCENT) ed energia consumata con ASCENT.

Quindi:

- **NO ASCENT:** DR decresce con l'aumentare della densità dei nodi (aumentano le collisioni).
- **CON ASCENT:** DR rimane costante, perché il numero di nodi attivi aumenta molto più lentamente del numero di nodi nella rete.

$ESR > 1$ sempre, quindi **ASCENT è sempre energeticamente più efficiente**.



9. Power management in generale

Mantenere la radio in sleep per più tempo possibile ma allo stesso tempo non far calare la QoS garantendo i requisiti di comunicazione dell'applicazione.

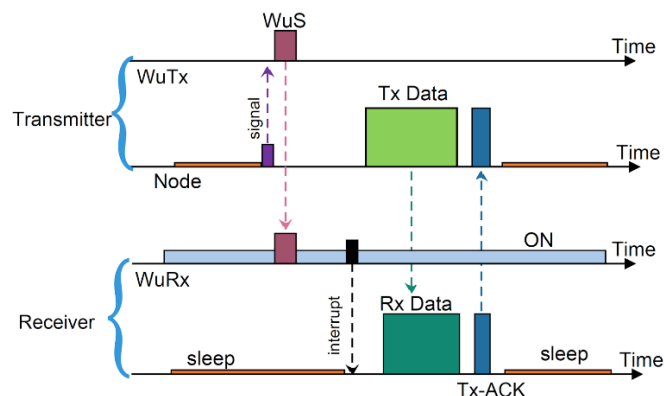
- **Low duty-cycle MAC protocols**
- **General sleep/wakeup schemes**

General sleep/wakeup schemes

On-demand

Tramite una wakeup radio, sveglia il destinatario con un segnale di wakeup (comprende l'indirizzo, per non svegliare altri nodi).

Dopo averlo ricevuto, il destinatario accende la data radio (che consuma di più) per ricevere i dati. Ci vuole tempo da quando riceve il segnale di wakeup a quando sarà pronto a ricevere i dati. Questa latenza può essere ridotta: quando iniziamo a ricevere il messaggio dall'hop precedente, inviamo il wakeup al next hop in modo quando il messaggio è stato completamente ricevuto, il next hop è pronto. Comporta un maggiore consumo energetico.

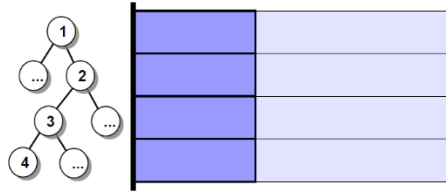


Aggiungere la WuR richiede del costo aggiuntivo per l'hardware.

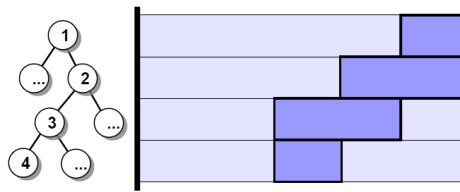
Scheduled rendez-vous

Il tempo viene diviso in slot (epoch). I nodi si coordinano riguardo uno slot temporale (rendez-vous) in cui sono tutti attivi.

- **Asynchronous:** Un nodo comunica quando vuole e gli altri si adattano (LPL e ContikiMAC).
- **Fully Synchronized:** All'inizio dello slot tutti i nodi attivi e comunicano. Dopo, tutti i nodi in sleep fino al termine dell'epoch. Richiede la sincronizzazione dei clock e presenta un alta probabilità di collisione.

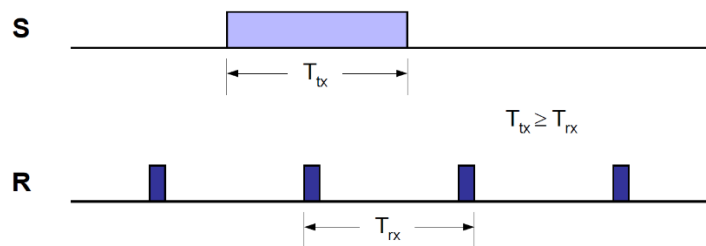


- **Fixed Staggered Scheme:** Coordinazione fatta a coppie padre-figlio: solo due livelli alla volta sono attivi. All'inizio solo le foglie e i loro padri sono attivi, successivamente sono attivi solo i padri delle foglie e i loro padri e così via.
 - **Avvantaggia la data aggregation**, messaggi inviati al padre solo quando il nodo ha ricevuto da tutti i figli.
 - **Tempo di attività di ogni nodo ridotto**, meno nodi sono attivi insieme → meno collisioni.
 - **L'activation time di un nodo dipende dalla sua posizione nell'albero**.
 - **Tempo di attività adattivo:** un figlio può indicare al padre di modificare il periodo di attività nel prossimo epoch.

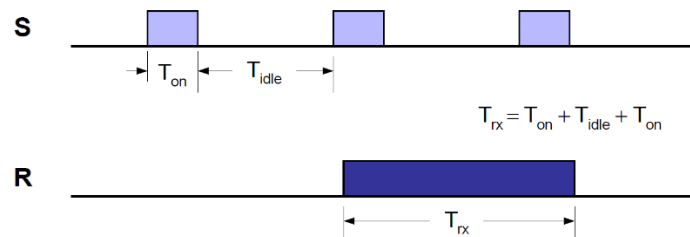


10. Burden on the sender e on the receiver

Sender.



Receiver



LPL – Low Power Listening

Prevede un funzionamento di questo tipo:

1. I nodi in sleep si **svegliano periodicamente**.
2. **CCA:** controllano se vi sono segnali (senza decodificare).
 - **Non c'è attività** → tornano in sleep.
 - **C'è attività** → rimangono attivi per ricevere il pacchetto.

Implementazione di **TinyOS** di LPL, basato su **burden on the sender**.

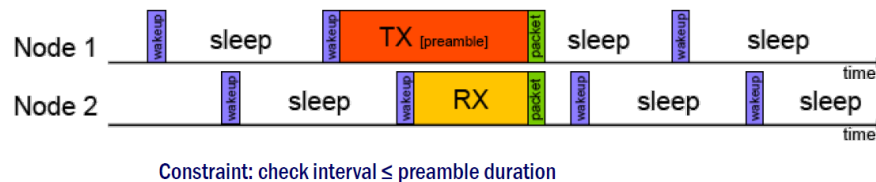
- I nodi **non si sincronizzano**.
- Il mittente invia un **preambolo** prima di ogni pacchetto per svegliare il receiver.

1. Node1 invia (preambolo + pacchetto) a Node2.
2. Node2 si **sveglia periodicamente** per TWakeUp e **controlla il canale**.

Il preambolo è lungo e contiene valori particolari che permettono di capire subito che si tratta di un **preambolo**.

3. Node2 nota che c'è una trasmissione in corso → **rimane attivo** per Trx.

Consideriamo $T_{tx} > T_{rx}$, poiché in T_{tx} è compresa sia la trasmissione del **preambolo** che del **pacchetto**.



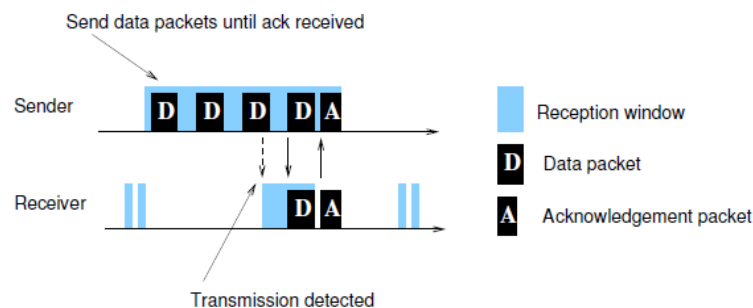
- TCheck_Interval → periodo tra due risvegli.
- Tpreamble → tempo necessario per trasmettere il **preambolo**.

Burden on the sender: Durante il **preambolo** il mittente consuma molto, ma non è un problema essendoci solo un trasmettitore. I receiver consumano poco visto che rimangono attivi poco.

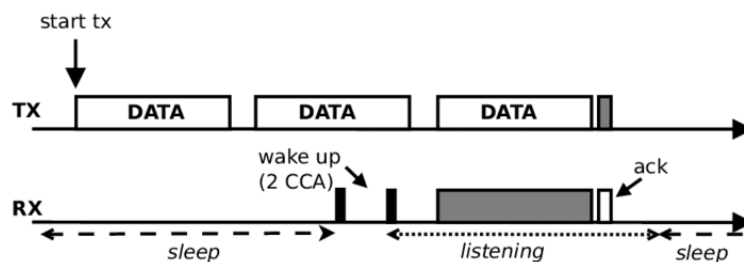
Problema: **preambolo** trasmesso in broadcast ed è identico per tutti → Tutti i potenziali ricevitori rimangono attivi fino al termine.

ContikiMAC

Il trasmettitore invia periodicamente D finché non riceve ACK. Quando un ricevitore riceve D, controlla se è diretto a lui: in caso negativo torna in sleep e non invia un ACK.



Tra un invio di una copia e l'altra, c'è un certo delay T_i per poter consentire al receiver di svegliarsi, elaborare e inviare l'ACK. Questa tecnica è **più efficiente di LPL**, perché viene **ridotta l'energia consumata da entrambi** perché il tempo di attività è minore per entrambi.

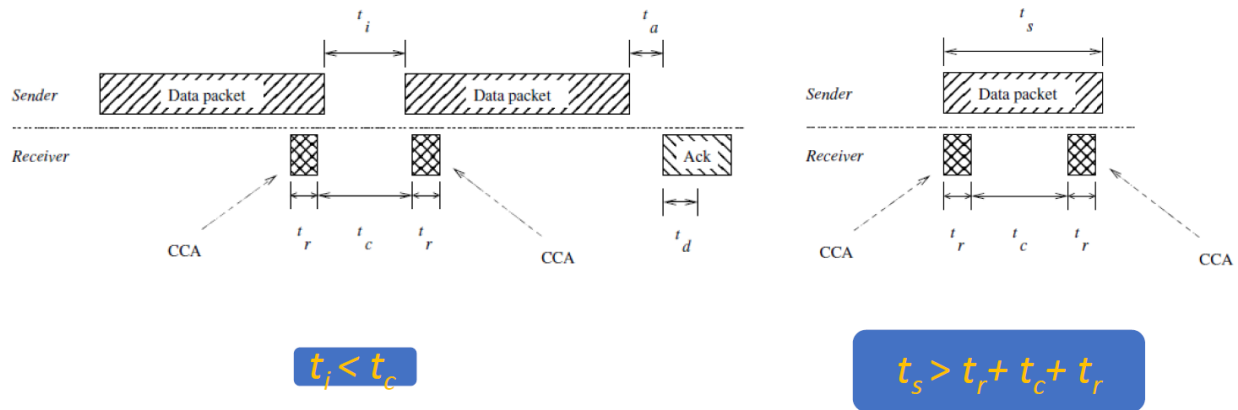


- T_c → Intervallo tra due CCA.
- T_r → Durata della CCA.
- T_i → Intervallo di tempo tra due trasmissioni di D.
- T_s → Durata della trasmissione di D.

Periodicamente il ricevitore:

1. **CCA**: Controlla il canale per un tempo T_r (durata fissa della CCA).
2. **Sleep**: per T_c .
3. **CCA**: Controlla il canale per un tempo T_r .
4. Se rileva una trasmissione, si mette in ascolto per ricevere tutto il pacchetto e per poi inviare un ACK.

Perché il ricevitore ripete questo ciclo di due CCA? Per **evitare falsi negativi**.



- $T_c > T_i \rightarrow$ assicura che almeno una delle trasmissioni sia rilevata da un CCA.
- $T_s > 2T_r + T_c \rightarrow$ assicura che D non sia trasmesso interamente tra due CCA consecutivi.

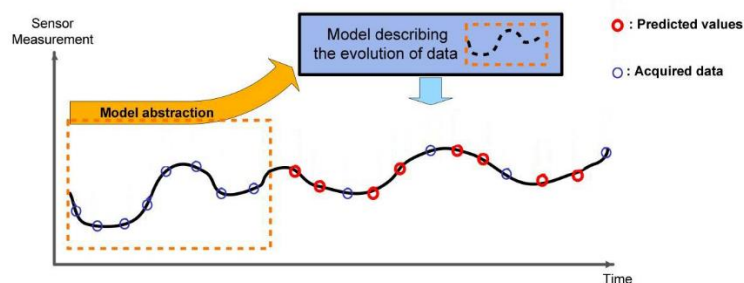
Ottimizzazione trasmettitore: prende nota di quale copia viene ricevuta (quella preceduta dallo ACK). Cercherà di sincronizzarsi sempre di più all'inizio del periodo di risveglio del ricevitore (per inviare meno copie).

Ottimizzazione ricevitore: nei periodi di silenzio (tra due D) va in sleep. Se un periodo di silenzio è maggiore di T_i allora non c'è nessun messaggio in corso (falso allarme) e torna in sleep.

Consumo energetico nei sensori e ASA

In certi sensori il consumo dell'elaborazione e campionamento è maggiore rispetto alla radio:

- **Hierarchical sensing:** Manteniamo sempre attivi gli LPS (economici, poco precisi) e attiviamo gli HPS (precisi, costosi) solo quando gli LPS misurano qualcosa.
- **Adaptive sampling:** Variamo la frequenza di campionamento in base al comportamento del fenomeno.
- **Model-based active sensing:** Si crea un modello del fenomeno. Invece campionare usa il modello per prevedere parte dei nuovi dati.



ASA – Adaptive Sampling

Minimizzare il rateo di campionamento $\rightarrow$ riduzione dei dati generati $\rightarrow$ meno attività computazionale.

Th di Nyquist $\rightarrow$ Sia F_{max} la frequenza massima del segnale, il rateo di campionamento $F_c \geq 2F_{max}$

ASA permette di **stimare dinamicamente** F_{max} e quindi **riadattare** F_c di conseguenza.

1. Da W campioni iniziali, stimiamo F_{max} e impostiamo $F_c = c * F_{max}$.

2. Definiamo le **soglie**:

- a. $F_{up} = \min \{ (1 + \delta) * F_{max} , F_c/2 \}$
- b. $F_{down} = (1 - \delta) * F_{max}$

3. Inizializziamo $h1 = h2 = 0$ // quante volte F_{curr} è sopra la soglia superiore o sotto la soglia inferiore.

4. Inizializziamo $i = W+1$ // numero totale di campioni.

while (true) :

5. i^o campione $\rightarrow$ al dataset.

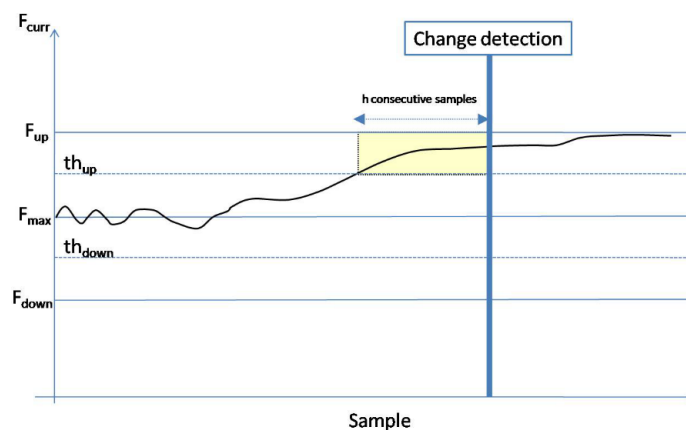
6. Stimiamo F_{curr} con gli ultimi W campioni ($i-W+1$, i). // Si usa sempre una finestra di W campioni.

7. **CUSUM test**:

- a. **Se** $|F_{curr} - F_{up}| < |F_{curr} - F_{max}|$ // F_{curr} più vicina a F_{up} rispetto che F_{max}
 - i. $h1++$, $h2 = 0$
- b. **Se** $|F_{curr} - F_{down}| < |F_{curr} - F_{max}|$ // F_{curr} più vicina a F_{down} rispetto che F_{max}
 - i. $h1 = 0$, $h2++$
- c. **Altrimenti** // La stima corrente F_{max} va ancora bene
 - i. $h2 = h1 = 0$

8. **Se** ($h1 > h$) **OR** ($h2 > h$) // Si aggiornano i valori

- d. $F_c = c * F_{curr}$ // si riadatta la frequenza di campionamento
- e. $F_{up} = \min \{ (1 + \delta) * F_{max} , F_c/2 \}$ // Calcolo soglie
- f. $F_{down} = (1 - \delta) * F_{max}$
- g. $F_{max} = F_{curr}$ // Nuova stima corrente



Parametri da configurare in base al processo osservato:

- $c \rightarrow$ quanto variare $F_{curr} \rightarrow c \geq 2$ (Th Nyquist).
- $\delta \rightarrow$ per calcolare le soglie.
- $h \rightarrow$ (**reattività**) determina quando modificare la frequenza.
 - Basso $\rightarrow$ reattivo ma poco stabile.
 - Alto $\rightarrow$ stabile, ma lento nel prendere decisioni (poco reattivo).
- $W \rightarrow$ (**precisione**) numero di campioni da analizzare nelle stime.
 - Piccolo $\rightarrow$ poco precisa, consuma poco.
 - Grande $\rightarrow$ più precisa, consuma di più ed è meno reattivo.

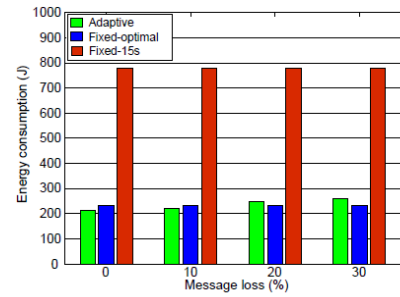
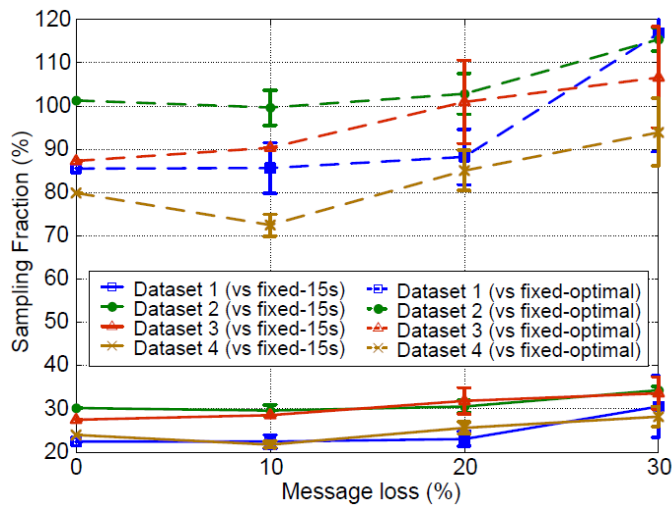
ASA Performance

Misurare il consumo con ASA e senza (rateo di campionamento fisso, supponiamo quello ottimo dato il dataset, nella realtà non sarebbe possibile, bisognerebbe sapere il futuro).

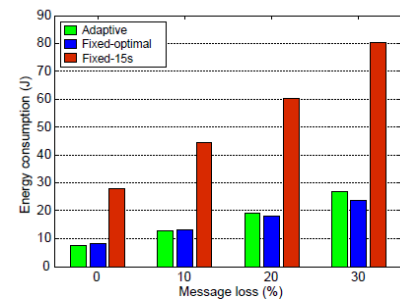
ASA permette di risparmiare molta energia e di produrre circa il 70% in meno di campioni.

La metrica è la **sampling fraction SF**: il rapporto tra il numero di campioni generati con ASA ed il numero di campioni generati con un rateo fisso.

$$\text{Sampling Fraction} = \frac{\text{Number of Samples with Adaptive Sampling}}{\text{Number of Samples with Fixed Sampling}}$$



Sensor Energy Consumption



Radio Energy Consumption

Harvesting aware – adaptive sampling

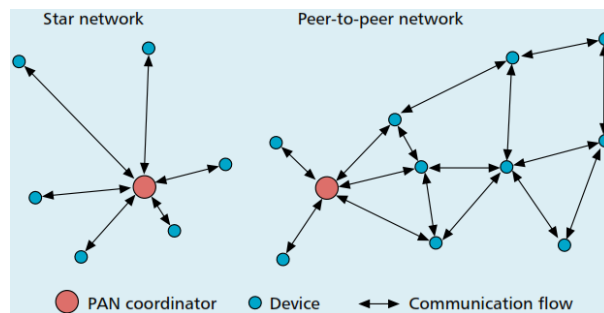
La frequenza di campionamento può anche essere adattata alla quantità di energia elettrica dentro al sensore.

IEEE 802.15.4

IEEE 802.15.4 in generale.

Standard la **comunicazione wireless efficiente** per PAN composte da dispositivi limitati.

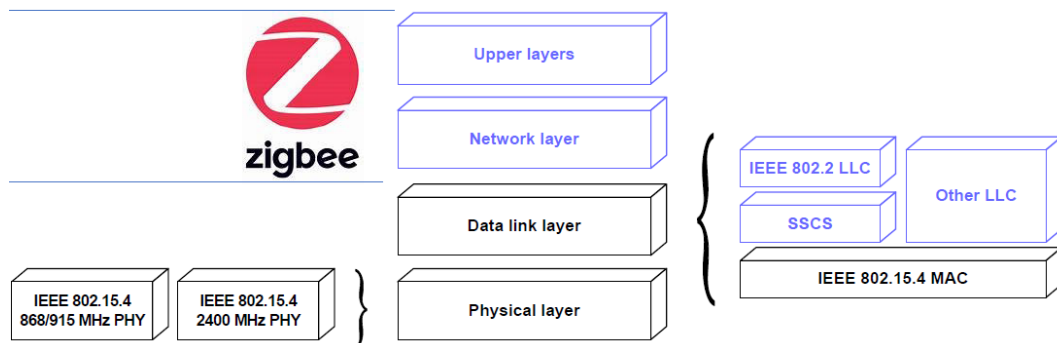
- **Frequenze e velocità** (27 canali):
 - Canale 0 (EU) → 868 MHz e 20 Kbps.
 - Canale 1-10 (USA) → [906 - 924] MHz e 40 Kbps.
 - Canale 11-26 (addizionali) → [2.4 – 2.483] GHz e 250 Kbps.
- **Range:** < 100 m.
- **Topologie:** stella o peer-to-peer (mesh).
- **Due modalità operative:** BE o NBE.



È lo standard di base su cui si costruiscono protocolli come: Zigbee, 6LoWPAN e WirelessHART.

Zigbee

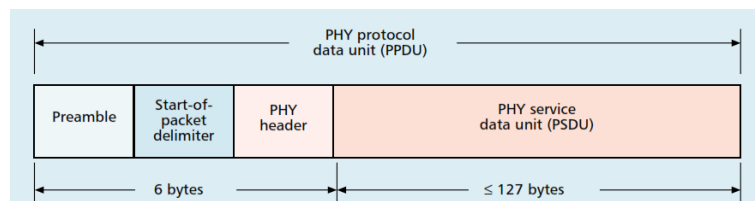
Standard costruito sopra 802.15.4, aggiunge i **livelli superiori** (network e applicazione/sicurezza) per dispositivi IoT.



PHY packet, MAC frame e il campo ADDRESS_INFO.

PHY Packet

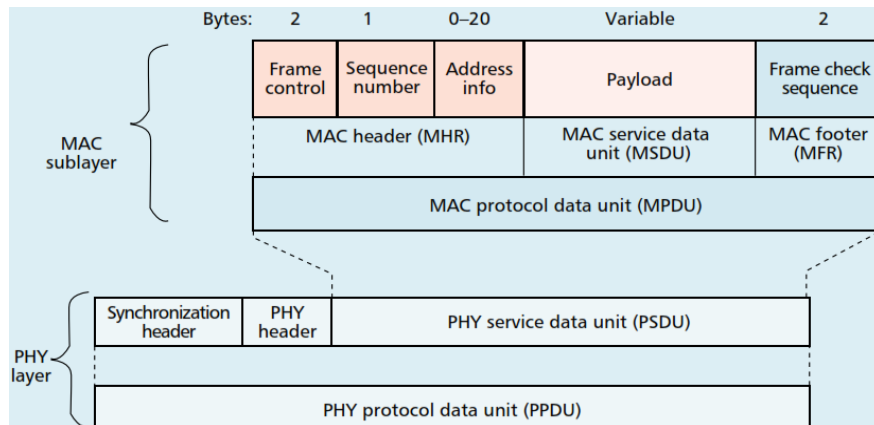
Il pacchetto PHY (PPDU) è l'unità trasmessa fisicamente via radio.



- **Preambolo [32 Bit]:** Per la rilevazione di una trasmissione.
- **Delimiter di inizio pacchetto [8 Bit]:** Per capire quando finisce il preambolo.
- **Header [8 Bit]:** Lunghezza del payload.
- **Payload:** il MAC frame, minore di 127 byte.

MAC Frame

Il pacchetto costruito dal livello MAC.



- **Frame control [2 Byte]**: Il tipo di frame (Data, ACK, Beacon)
- **Sequence Number [1 Byte]**: Per tenere traccia dei duplicati.
- **ADDRESS_INFO [20 Byte]**
- **Payload** [102 con indirizzi estesi, 114 con indirizzi corti]
- **FCS [2 Byte]**: CRC e controllo errori.

ADDRESS_INFO [20 Byte]

Contiene gli indirizzi (non IPv6) a 16 bit o a 64 bit del mittente e del destinatario, permette il routing a livello MAC.

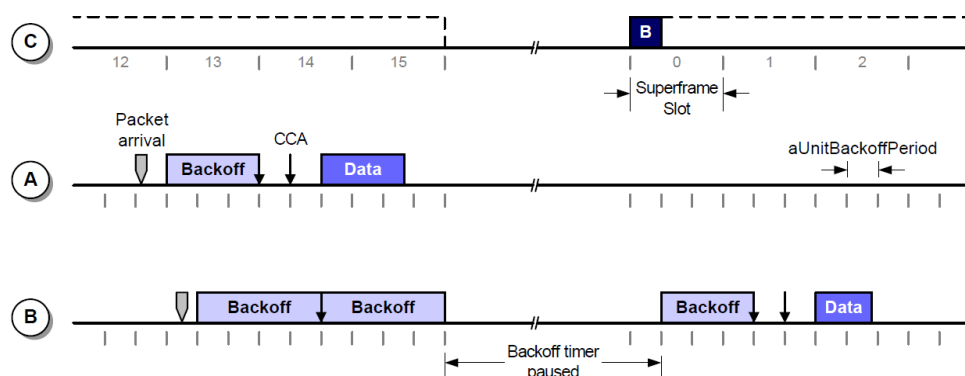
- **Corto [16 bit]**: Assegnato dinamicamente dal PAN Coordinator, quando entra nella rete.
- **Esteso [64 bit]**: 24 bit più significativi assegnati da IEEE, gli altri dal costruttore.

Considerando l'indirizzamento esteso troviamo **20 byte per l'indirizzamento**.

- **16 byte** per l'indirizzo sorgente e destinazione.
- **4 byte** per l'indirizzo PAN sorgente e destinazione.

802.15.4 differenza tra BE e NBE.

a. Draw a beacon interval with contention



b. Can nodes start transmission at any time?

- **BE → No**, deve sempre essere allineato agli slot.
 - **NBE → Si**
- c. CSMA/CA async, and exponential backoff

Secondo 802.15.4 una rete può operare in due modi:

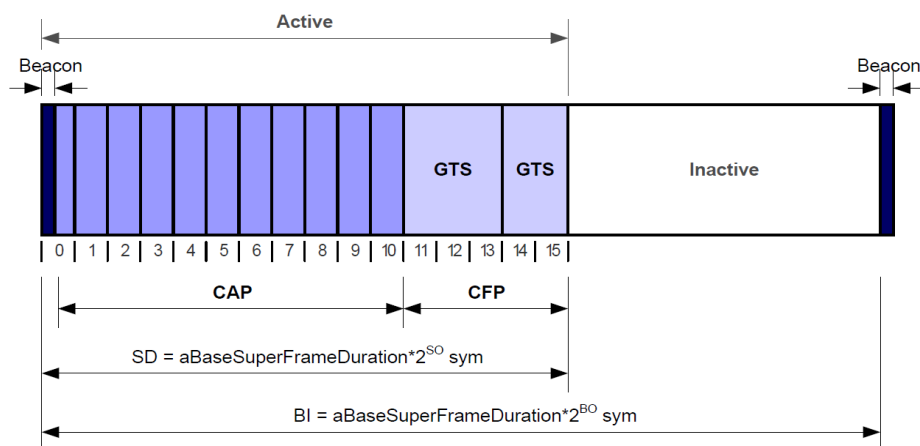
- Non-beacon enabled.
- Beacon enabled.

Beacon enabled

Nella rete è presente un PAN coordinator che invia **periodicamente** in **broadcast** un MAC frame di tipo “beacon”.

Questa comunicazione è di tipo rendez-vous (tutti i nodi attendono un momento comune in cui comunicare), l’inizio del rendez-vous è la ricezione del beacon.

1. **Beacon Interval (superframe duration):** l’intervallo tra due trasmissioni consecutive del beacon.
2. **Superframe:** lo spazio tra due beacon consecutivi:
 - **Parte attiva:** 16 slot temporali di durate diverse.
 - **Parte inattiva:** La parte rimanente in cui tutti spengono la radio.



- **CFP (Collision Free Period)** : composto dai Granted Time Slot (GTS).
 - In essi non possono avvenire collisioni poiché gestiti con **TDMA** (solo nodo può trasmettere).
 - Assegnati in maniera centralizzata dal coordinator ed è scritto nei beacon. Durante il CAP un nodo può richiedere un GTS al coordinator. Il nodo viene a conoscenza se la sua richiesta è stata accettata leggendo i prossimi beacon.
- **CAP (Contention Access Period):** i restanti slot non assegnati a nodi specifici, possono avvenire collisioni.

NB: Il GTS potrebbe non essere presente, mentre il CAP è sempre presente.

Gestire le collisioni nel CAP → **Slotted CSMA/CA**. Indichiamo con questo **colore** i parametri dell'algoritmo. Tutta questa procedura viene eseguita durante il CAP.

1. Il nodo attende l'inizio del prossimo slot del CAP e inizializza:

- $BE = \text{MIN_BE}$
- $CW = 2$

La procedura di sicuro non verrà eseguita tutta entro uno slot del CAP, ma comunque deve rimanere entro il superframe. Se va oltre, si sospende e si continua al prossimo superframe.

2. $\text{Backoff_Timer} = \text{rand}(0, 2^{BE} - 1)$

3. $\text{sleep}(\text{Backoff_Timer})$

4. **1° CCA**

- **Libero**
 - $CW -= 1$
- **Occupato**
 - $NB += 1$ // Numero di tentativi
 - $CW = 2$
 - $BE = \min(BE + 1, \text{MAX_BE})$ // Backoff window **raddoppiata**
 - **If** $NB > \text{MAX_NB}$ → $\text{abort}()$ // Scartato, per max num. di fallimenti
 - **Else** → Riproviamo → Punto 2

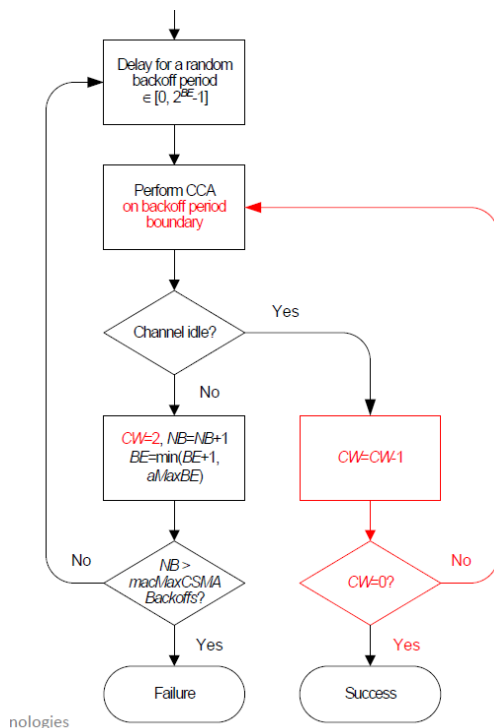
5. **2° CCA**

- **Libero** {...} // Stesso codice.
- **Occupato** {...}

6. **If** $CW == 0$ → Trasmissione.

7. Se ACK abilitato:

- Attende ACK per un periodo.
- Se non riceve entro il periodo → $NT++$.
- **If** $NT > \text{MAX_NT}$ → $\text{abort}()$
- **Else** → Riproviamo → Punto 6

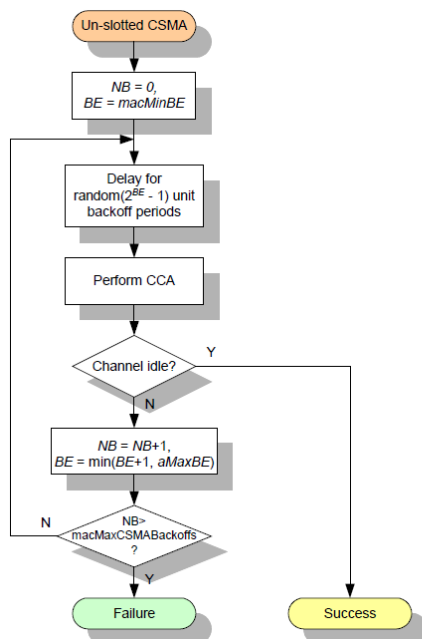


At each trial the backoff-window size is doubled

Only a limited number of attempts is permitted (*macMaxCSMABackoffs*)

Non-beacon enabled

Non deve attendere l'arrivo di un beacon (la trasmissione può iniziare in qualsiasi momento). Il nodo effettua un solo CCA per regolare l'accesso al canale. Per regolare l'accesso al canale si usa **Un-Slotted CSMA/CA**.



At each trial the backoff-window size is doubled

Only a limited number of attempts is permitted (*macMaxCSMABackoffs*)

Se un messaggio non viene ricevuto correttamente dalla destinazione:

- **ACK abilitato**: viene trasmesso il pacchetto se non viene ricevuto un ACK dopo un certo periodo di tempo.
- **ACK disabilitato**: il pacchetto viene inviato solo una volta, indipendentemente dal successo o meno.

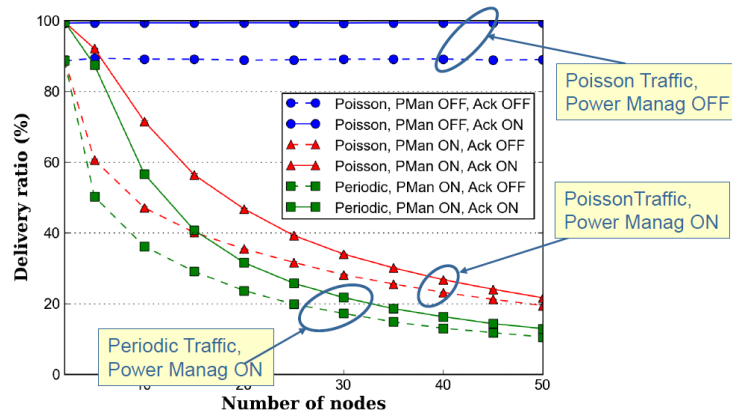
Perchè due CCA nella BE mentre in NBE uno solo?

In mancanza di slot temporali, è **inutile** fare più CCA nella NBE: il canale può cambiare in ogni istante. Mentre nella BE gli slot sono sincronizzati, e c'è maggiore rischio che più dispositivi inizino la trasmissione nello stesso istante.

Analisi prestazioni di 802.15.4

Packet Loss

Assumiamo che la rete operi in **BE** e una topologia a **stella**. Concentriamoci sulla fase CAP e misuriamo il DR con l'aumentare del numero di nodi. Supponiamo 10% di perdita.



- **Poisson, ACK ON, PMan OFF** → DR = 100%.
- **Poisson, ACK OFF, PMan OFF** → DR circa 90%

Il canale non è ideale quindi i pacchetti affrontano errori di trasmissione. Noi abbiamo supposto che la message loss sia pari al 10%. Il prezzo da pagare per non mettere lo ACK.

- **Poisson, ACK ON, PMan ON** → Se $n=50$, DR circa 20%.

Introducendo la PMan ci saranno periodi di inattività (radio spenta). Il beacon in questo caso è un punto di sincronizzazione per accendere la radio. Quando un nodo riceve un pacchetto dei livelli superiori durante la parte inattiva, viene memorizzato in un buffer e trasmesso quando il dispositivo ritorna attivo.

DR decresce drasticamente, perché all'aumentare dei nodi cresce anche il numero di pacchetti "ricevuti" durante il periodo di inattività e quindi il numero di pacchetti inviati appena ritornano attivi (aumento di collisioni).

- **Poisson, ACK OFF, PMan ON** → Se $n=50$, DR circa 20%.
- **Periodico, ACK ON, PMan ON** → Se $n=50$, DR circa 20%.

La competizione è massima, perché tutti i nodi nella rete hanno il **periodo di attività sincronizzato** e quindi vogliono trasmettere il pacchetto allo stesso momento.

- **Periodico, ACK OFF, PMan ON** → Se $n=50$ → DR circa 15%.

Ci sono alcune possibili risposte:

- **CSMA**: non è scalabile → Analisi dei parametri.
- **Beacon-Mode**: Il beacon periodico comporta che tutti i nodi si sincronizzano e ciò aumenta le collisioni.

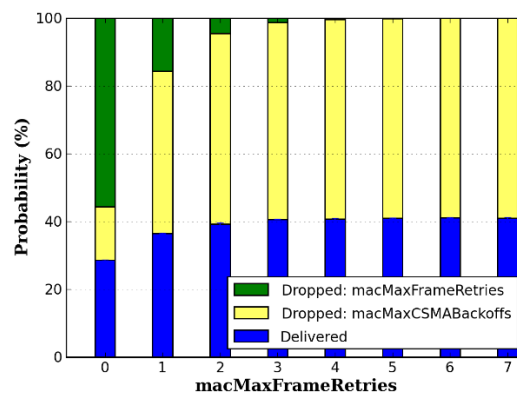
Analisi dei parametri di CSMA/CA

Variare ogni parametro e vedere quanto impatta.

Parameter	2003 release	2006 release	Notes
<i>macMaxFrameRetries</i>	3 (<i>aMaxFrameRetries</i>)	0÷7 Default: 3	Max number of re-transmissions
<i>macMaxCSMABackoff</i>	0÷5 Default: 4	0÷5 Default: 4	Max number of backoff stages
<i>macMaxBE</i>	5 (<i>aMaxBE</i>)	3÷8 Default: 5	Maximum Backoff Window Exp.
<i>macMinBE</i>	0÷3 Default: 3	0÷7 Default: 3	Minimum Backoff Window Exp.

Indici di performance: DR, latenza media e consumo energetico medio per messaggio.

maxMaxFrameRetries (massimo numero di ritrasmissioni)



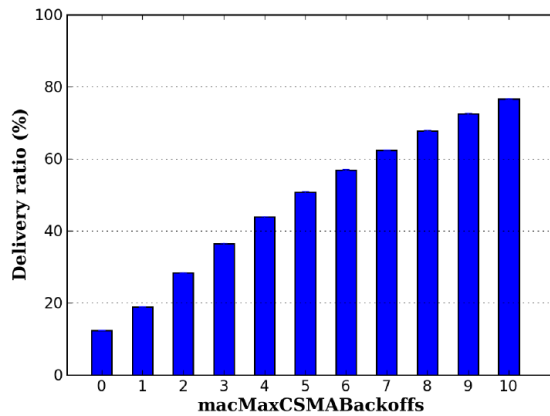
Consideriamo un intervallo tra 0 e 7 per il numero di ritrasmissioni, questi valori sono quelli indicati dallo standard:

- Da 0 a 1 → incremento significativo del DR.
- Da 1 a 2 e da 2 a 3 → miglioramento piccolo.
- Da 3 a 4 in poi → nessun miglioramento, solo spreco di energia e aumenta il delay.

macMaxCSMABackoff (massimo numero di fallimenti)

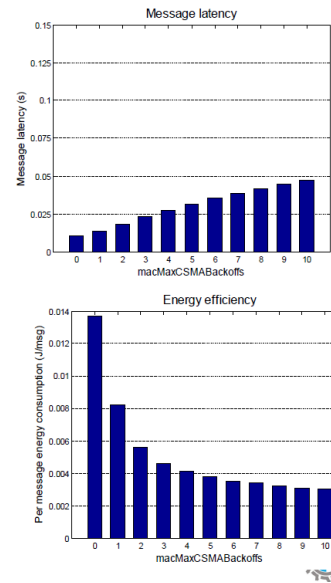
Aumentiamo fino ad un massimo di 10.

macMaxCSMABackoffs: 0-5 (default 4)



Communication Technologies

38



- **DR:** aumenta in modo lineare.
- **Latenza dei messaggi:** aumenta.
- **Energia totale:** aumenta perché trasmettiamo più pacchetti e perché controlliamo il canale per più tempo.
- **Energy per packet** (rapporto tra l'energia totale ed il numero di pacchetti trasmessi correttamente) decresce.

Aumentiamo il numero massimo di fallimenti → si prova per più tempo → consumiamo più energia.

Ma essendo l'energy per packet un rapporto tra due grandi quantità, esso decresce, quindi l'efficienza energetica aumenta (se non aumentiamo il numero di pacchetti, consumiamo molta energia per pochi pacchetti ed il valore del ratio aumenta).

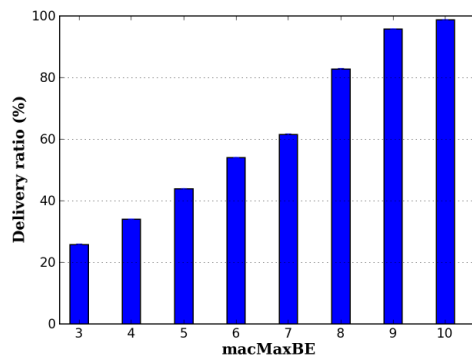
macMaxBE

Aumentiamo fino ad un massimo di 10.

- **DR:** Aumenta di molto.
- **Latenza dei messaggi:** aumenta.
- **Energia totale:** aumenta.
- **Energy per packet** decresce.

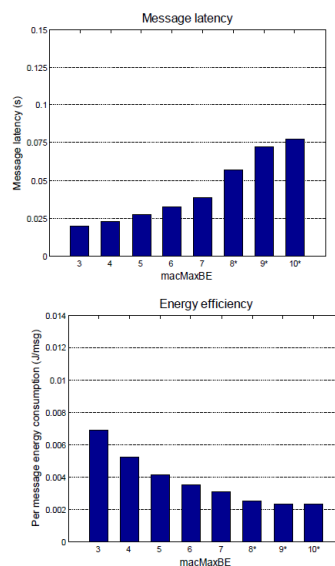
macMaxBE: 3-8 (default 5)

macMaxCSMABackoffs ≥ macMaxBE – macMinBE



tion Technologies

39

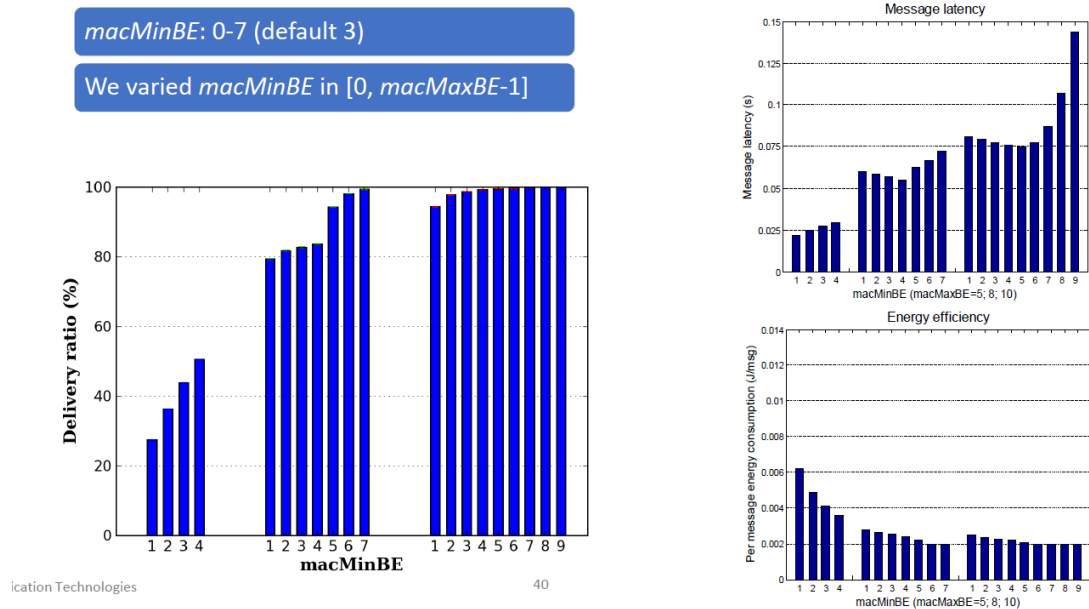


macMinBE

Consideriamo tre set di esperimenti perché deve essere coordinato con macMaxBE. Nel primo caso è compreso tra [1, 4], nel secondo tra [1, 7] e nel terzo tra [1, 9].

Il macMinBE è il parametro che ha l'influenza maggiore sul DR → è ragionevole utilizzare un BE grande.

Vengono generati valori di BE in una finestra grande → i tentativi di trasmissione sono diffusi in un grande intervallo di tempo → diminuisce le probabilità che il canale wireless sia occupato → aumenta la probabilità di trasmissione.



Come si misura il packet loss rate?

Stimata tramite il rapporto tra pacchetti inviati dai nodi e ricevuti dal sink.

Bluetooth

Bluetooth

	Bluetooth V2.1	Bluetooth Low Energy
Standardization Body	Bluetooth SIG	Bluetooth SIG
Range	~30 m (class 2)	~50 m
Frequency	2.4–2.5 GHz	2.4–2.5 GHz
Bit Rate	1–3 Mbit/s	~200 kbit/s
Set-Up Time	<6 s	<0.003 s
Voice Capable?	Yes	No
Max Output Power	+20 dBm	+10 dBm
Modulation Scheme	GFSK	GFSK
Modulation Index	0.35	0.5
Number of Channels	79	40
Channel Bandwidth	1 MHz	2 MHz

BLE ha un **bitrate minore**, (quindi **range maggiore**) e può utilizzare 40 canali, mentre il Bluetooth può utilizzarne 79.

- I primi tre per il beaconing.
- Gli altri per scambiare dati.

Il BLE può essere in due modalità operative:

- **Advertising mode:** Vengono inviati i beacon seguendo un approccio di tipo master-slave.
- **Communication mode:** Vengono scambiati i dati tra i dispositivi seguendo una comunicazione broadcast.

BLE può supportare le seguenti topologie:

- **Hub and Spoke:** hub centrale e tutti gli altri dispositivi sono connessi solo ad esso.
- **Mesh topology:** i dispositivi sono interconnessi e ognuno può comunicare direttamente con tutti gli altri senza un coordinatore. Il coordinatore avvia la rete, ma non è un punto centrale come un hub.

Una rete basata su bluetooth può connettersi direttamente ad internet dalla versione 4.1 in poi. Ma prima serviva un gateway, ovvero un dispositivo (come uno smartphone) che faceva da tramite.

Dalla versione 4.1 in su, sopra il Bluetooth viene implementato un livello IPv6; quindi, un dispositivo di questo tipo è considerato nativo IPv6. Inoltre, per adattarsi, il WiFi ha introdotto una versione per le comunicazioni low power: **WiFi HarLow e LPWA**.

RPL

Tassonomia dei protocolli di routing

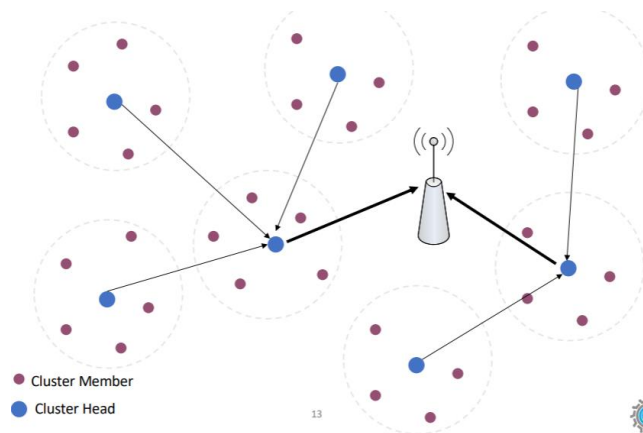
- **Flat routing:** No gerarchia e si sfrutta il **flooding**
- **Hierarchical routing:** C'è una gerarchia ed è indicato per la **data aggregation**:
- **Location-based routing:** Ogni nodo conosce la sua posizione e la sfrutta per il routing.

Routing gerarchico

Cluster-based (LEACH)

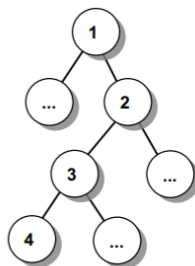
Nodi organizzati in clusters, in ogni cluster c'è un CH. Ogni nodo sceglie come proprio CH (quindi anche il proprio cluster) quello più vicino, in modo da minimizzare l'energia spesa per la comunicazione (RSSI o tramite GPS).

Il CH riceve i messaggi dai propri nodi e inoltra verso il sink attraverso gli altri CH.



Tree-based

Viene creato un albero logico. Ogni nodo, quando deve inviare un pacchetto, lo invia al proprio padre.



Tipologie di pacchetti in RPL.

DIO – DODAG Information Object

Messaggio **broadcast** per **pubblicizzare un DODAG**. Permette ad un nodo di:

- Scoprire un'istanza di RPL e impararne i parametri di configurazione.
- Selezionare un set di padri e il padre preferito.
- Unirsi o mantenere aggiornate le informazioni sul DODAG.

DIS

Messaggio **unicast o broadcast** per sollecitare l'invio di un DIO da un nodo o per sondare la presenza di vicini in altri DODAG adiacenti. Quando un nodo riceve un DIS, risponde con un DIO.

DAO

Messaggio **unicast** per permettere ai padri di scoprire gli indirizzi conosciuti dai figli. Quando un DAO è ricevuto dalla radice, viene stabilito un percorso dalla radice al nodo mittente originale.

Costruzione del DODAG

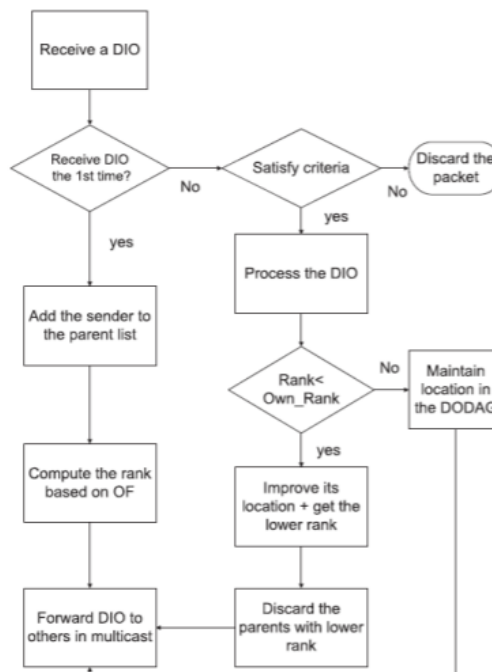
Upward → route dai nodi alla radice (route di default).

1. La radice invia **periodicamente** i DIO in **broadcast** per permettere ai nodi di annunciare (DODAG ID, il proprio rank e la OF nel campo opzioni del DIO).

La trasmissione è regolata dal **Trickle Timer**: adatta la frequenza in base alla stabilità della rete: più frequente in caso di cambiamenti, meno frequente in condizioni stabili.

Il DIO è ricevuto da tutti i nodi. Ricevendolo, ogni nodo conosce i propri vicini (nodi ad un hop di distanza).

2. **Primo DIO:**
 - a. Aggiunge il mittente alla lista dei padri candidati.
 - b. Calcola il proprio rank utilizzando la OF (specificata nel DIO) e il rank del mittente.
 - c. Sceglie il mittente come padre preferito (l'unico nella lista).
3. **Altrimenti:**
 - a. Se il DIO non soddisfa i requisiti → Scarta.
 - b. *Se il Rank del mittente è inferiore al Rank del nodo:*
 - i. Aggiunge il mittente alla lista dei padri candidati.
 - ii. Calcola il rank **che avrebbe** scegliendo il mittente come padre.
 - iii. *Se è meglio di quello attuale* → diventa il nuovo padre preferito.
4. **Inoltra il DIO** (aggiornando Rank se cambiato ed eventuali altre informazioni).



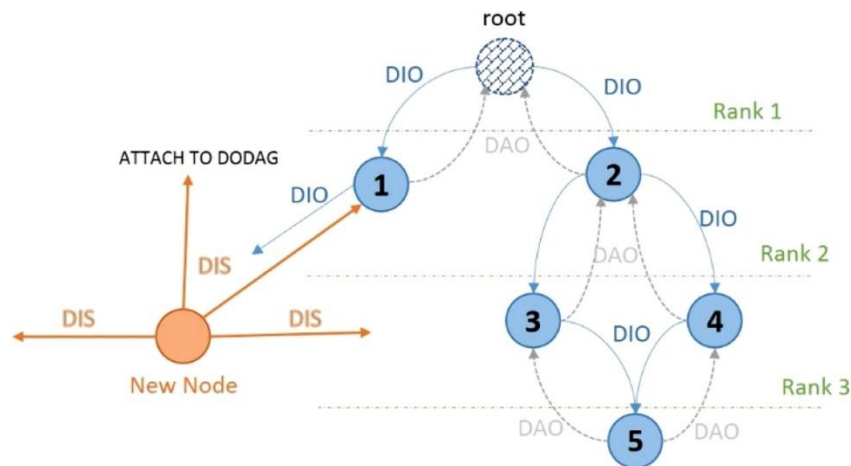
Se un nodo rileva che il proprio Rank è inferiore a quello del suo padre preferito, si verifica una violazione delle regole di RPL. Questo può accadere, ad esempio, se il genitore ha aumentato il proprio Rank a causa di **cambiamenti nella topologia della rete** o di **degradazione del collegamento tra il padre e il figlio**.

RPL ha fa **manutenzione** per adattarsi ai cambiamenti nella topologia della rete:

- **Riparazione Locale:** Se un nodo perde il collegamento con il proprio padre preferito, può selezionare un nuovo genitore tra la lista di candidati disponibili creati all'inizio.
- **Riparazione Globale:** Da fare quando i guasti locali sono molti. La radice forza tutti i nodi a ricalcolare i propri Rank e quindi a ricostruire il DODAG.

Routing Downward: differenza tra storing e non-storing mode. Qual è quello più indicato nello IoT e perché?

Downward → route dalla radice verso i nodi.

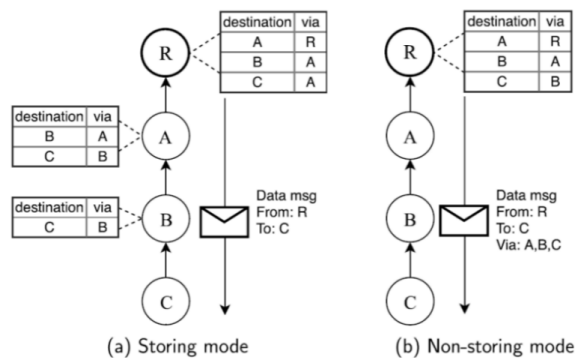


Il DAO è **inviato unicast al padre**. Contiene tutti gli indirizzi conosciuti dal nodo, in modo che li comunichi al padre. Si propagano fino alla radice.

RPL utilizza i DAO con due modalità:

- **Storing Mode:** Ogni nodo mantiene una tabella di routing per i propri discendenti.
- **Non-Storing Mode:** Solo la radice mantiene la tabella di routing completa. Il DAO va direttamente alla radice, che utilizza queste informazioni per costruire rotte source-routed attraverso il source routing di IPv6.

Per lo IoT la modalità più indicata è la **non-storing**, perché i dispositivi hanno **memoria limitata**.



I DAO possono includere opzioni come:

- **Target:** l'indirizzo del nodo che invia il DAO.
- **Transit Information:** informazioni sul percorso verso il nodo target.
- **DAO Sequence Number:** per tracciare le versioni dei messaggi DAO.

RPL (cos'è un'istanza, generalità ecc.)

Algoritmo di routing di tipo **Tree-based** per **dispositivi limitati**:

- Ogni nodo sceglie un **genitore preferito**, quindi il **percorso di routing verso la radice** forma un **albero**.
- **Da ogni nodo verso la radice, c'è un unico percorso.**

- Permette a un nodo di mantenere **più genitori candidati** (backup), **robustezza migliorata**.

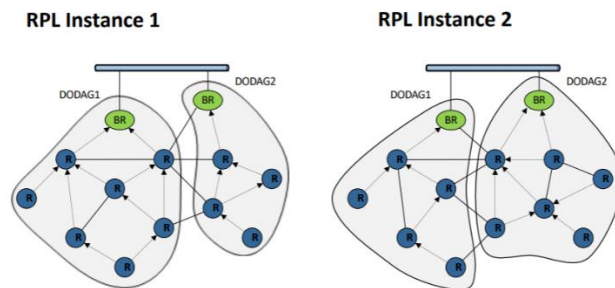
RPL si basa su una struttura ad albero detta DODAG (Destination-Oriented Directed Acyclic Graph). In questo albero, un nodo ha più padri e le route sono calcolate mediante un sistema simile al distance vector.

Il DODAG ha origine da un nodo radice, in genere il **BR**.

Istanza di RPL

In una LLN possono essere eseguite diverse istanze di RPL (identificate da un `RPLInstanceID`), per soddisfare diversi requisiti di routing.

Un istanza può gestire più DODAG, i quali condivideranno le stesse metriche e vincoli. Un nodo può appartenere a più istanze contemporaneamente ma solo ad un DODAG per ogni istanza.



Rank

Intero che **rappresenta la posizione del nodo** nel DODAG. Il rank aumenta in direzione downstream (dalla radice verso le foglie). Il rank NON misura il costo del percorso per andare verso la radice, ma solo la posizione del nodo nell'albero.

OF – Funzione Obiettivo

Il rank è il risultato dell'elaborazione di molte metriche:

- **Ritardo.**
- **Larghezza di banda.**
- **Affidabilità.**
- **Energia nella batteria:** specifica, tramite 2 bit quanta autonomia rimane.
- **Overload:** Overload della CPU e memoria rimanente.
- **Hop count dalla radice.**

La OF specifica:

- Come calcolare il rank.
- Come calcolare e pubblicizzare il path cost.
- Come selezionare il padre preferito (quando, chi e come).

Come gestire l'albero:

- **Max-depth rule:** Un nodo non può avere come genitore un nodo con rank più alto di `node_minimum_rank + DAGMAXRankIncrease`. Questo permette di prevenire un count-to-infinity quando un loop avviene.
- **Sub-DODAG Poisoning:** Un nodo può "avvelenare" un sub-DODAG pubblicizzando un rank di valore `INFINITE_RANK`.

I loop sono difficili da prevenire in modo sicuro, per questo RPL ha tecniche per rilevare e poi risolvere i loop. Ci sono varie tecniche per fare **loop detection**:

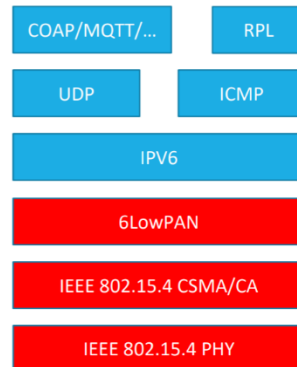
- **Data Path Validation:** Vengono inserite delle informazioni nel pacchetto per assicurarsi che esso si muova sempre in direzione forward. Un pacchetto può essere inviato ad un fratello (rank uguale) solo una volta. Se

un altro nodo cerca di inviare lo stesso pacchetto ad un fratello, il pacchetto viene scartato, perché è stato rilevato un loop.

TSCH

Come si “arriva” a TSCH da 802.15.4?

L'architettura proposta da 802.14.5:



Il problema dietro a 802.15.4 è in dovuto allo **slotted CSMA/CA** e alla **mancata gestione di interferenze e multipath fading**:

- **Latenza non prevedibile.**
- **Poca Affidabilità** → CSMA/CA → La BE provoca alta contenzione → aggravato dai parametri standard di CSMA/CA, non adatti alle WSN → Si può mitigare usando parametri non standard a discapito della latenza e del consumo energetico.
- **No frequency hopping** → multipath fading e le interferenze urtano molto l'affidabilità e non risolvibile senza un nuovo protocollo.

TSCH fa parte di 802.14.5 come modalità operativa opzionale. TSCH usa ancora i beacon ma solo per sincronizzare le informazioni tra i nodi (formation process).

Perché non si può usare 802.15.4 per applicazioni industriali?

Le applicazioni industriali hanno **stringenti requisiti di affidabilità** (situazioni gravi in tempo reale) e di **throughput**.

- **Classe 0:** Emergenza (wired)
- **Classe 1:** Closed Loop regulatory control.
- **Classe 2:** Closed Loop supervisory control.
- **Classe 3:** Open loop control.
- **Classe 4:** Alerting.
- **Classe 5:** Logging and notification.

Ora l'utilizzo del WiFi nei dispositivi è massivo ma comunque non possiamo rinunciare ai requisiti di affidabilità e di throughput.

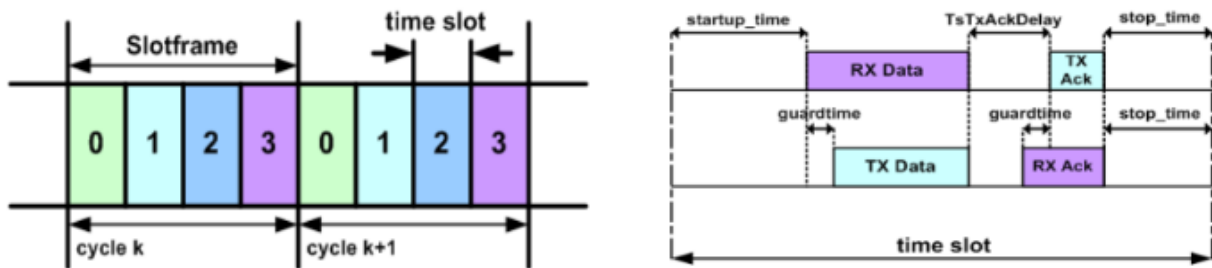
TSCH obiettivi, channel hopping e frequenze

TSCH fornisce un **livello data-link adatto ai loro requisiti**:

- **Time-slotted access:** latenza prevedibile e larghezza di banda garantita.
- **Multichannel:** permettere a più coppie di nodi di comunicare contemporaneamente.
- **Frequency hopping:** Mitigare l'effetto del multipath fading e delle interferenze.

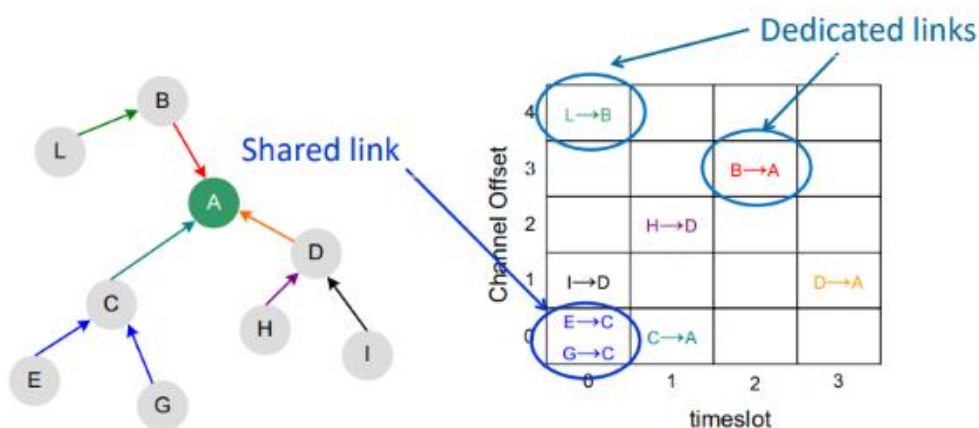
TSCH Slotframe

Il tempo diviso in slot di **durata fissa**, definiti in modo che la ricezione del pacchetto e la ricezione del relativo ACK sia possibile.



TSCH Link

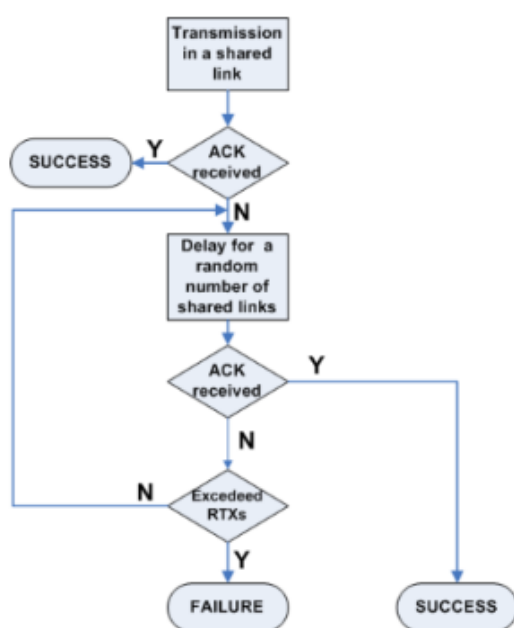
Ogni slot viene assegnato ai nodi secondo una certa politica:



Due tipologie di link in TSCH:

- **Dedicato:** Per nodi che comunicano spesso. Accesso diretto al canale, non serve CCA.
- **Condiviso:** Per nodi che comunicano raramente. Serve **TSCH CSMA/CA** per regolare l'accesso.

Il **TSCH CSMA/CA** è diverso dal CSMA/CA classico (stesso nome perché hanno la stessa funzione, gestire le collisioni).



Backoff mechanism

is activated only after the node has experienced a collision (to avoid repeated collisions)

Backoff unit duration

TSCH CSMA/CA: a shared slot
802.15.4 CSMA/CA: 320 μsec

Clear Channel Assessment (CCA)

CCAs are not used to prevent collisions among nodes, but to avoid transmitting a packet if a strong external interference is detected

Packet dropping

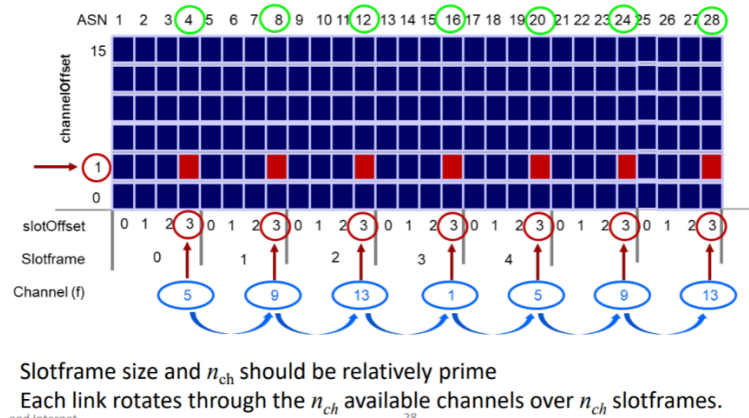
a packet is dropped only if it reaches the maximum number of retransmissions

Rispetto al CSMA/CA originale: questo algoritmo è **reattivo**: prima si trasmette, se non viene ricevuto lo ACK (si assume una collisione) si salta lo slot.

Cos'è il frequency hopping (formula) con timeslot e lo schema dei channel offset.

Tecnica che consente di cambiare frequenza di trasmissione, in maniera pseudo-randomica o deterministica. **Pacchetti consecutivi o ritrasmissioni sono eseguiti a frequenze diverse → mitiga interferenze e multipath fading.**

$$f = F \{ (ASN + chOf) \mod n_{ch} \}$$



- **SlotFrameSize** → Quanti slot presenti in un frame.
 - Nell'esempio 4.
- **f** → Canale fisico da usare in questo slot.
- **ASN** → Numero totale di slot passati da quando la rete è stata avviata.
- **Nch** → Numero di canali fisici utilizzati.
 - Nell'esempio 16.
- **F** → Mappa il valore numerico dell'indice in una frequenza radio (canale fisico).
 - Nell'esempio è una funzione di identità (non fa nulla).
- **chOf** → Indice numerico assegnato ad ogni link logico (e.g. A→B).
 - Nell'esempio 4.

Come fare ad usare tutte le frequenze disponibili? per fare in modo che **f** possa assumere tutti i valori tra **0** e **Nch**, basta che **Nch** e **SlotFrameSize** siano **coprimi**.

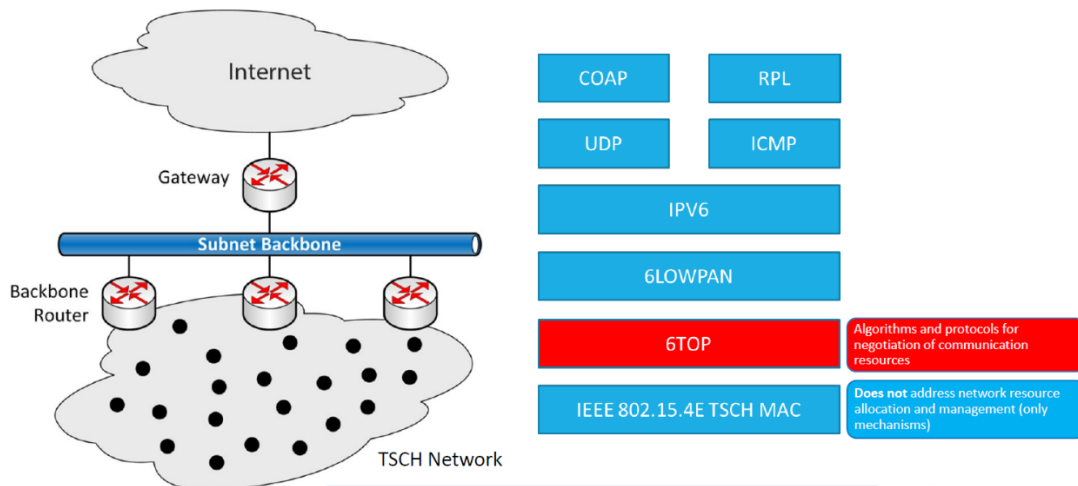
Architettura 6TiSCH

Se sostituiamo CSMA/CA con TSCH alcune assunzioni dei livelli superiori (IPv6) non sono più valide:

- **Non definisce il riutilizzo delle celle** assegnate ai nodi: assume che avvenga ai livelli superiori.
- **Non permette la pianificazione ai percorsi multi-hop gestiti da RPL**, meccanismi per adattare le risorse allocate tra nodi vicini ai flussi di traffico dati.
- **Non permette un trattamento differenziato dei pacchetti.**
 - Pacchetti dati generati a livello applicativo.
 - Messaggi di segnalazione necessari a 6LoWPAN e RPL per individuare i vicini e reagire ai cambiamenti di topologia.

6TiSCH è l'architettura per far funzionare IPv6 su reti basate su TSCH. Include le seguenti componenti:

- **RPL**
- **6LoWPAN**
- **6TOP** (negoiazione delle risorse)



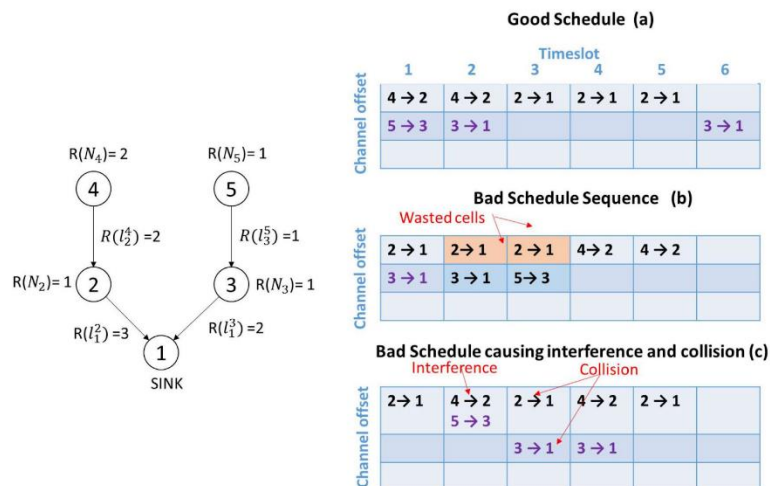
6TOP e come avviene una negoziazione

Livello di adattamento di 6TiSCH e compie lo **scheduling delle celle TSCH**. Tipologie di scheduling supportate:

- **Centralizzato**: I nodi inviano informazioni al PCE il quale calcola lo schedule ottimo per ogni nodo.
 - Es: AMUS.
- **Statico**: I progettisti decidono lo schedule per ogni nodo.
- **Decentralizzato Autonomo**: ogni nodo calcola il suo schedule in base alle informazioni locali che conosce, senza comunicare con gli altri nodi.
- **Decentralizzato Collaborativo**: ci sono accordi tra coppie di nodi per decidere lo slot in cui comunicare. Scambio di messaggi tra i nodi, molto overhead in caso di dinamicità.
 - Es: 6P.

AMUS

Algoritmo di scheduling **centralizzato** in cui le risorse sono riservate lungo tutto il percorso tra due nodi.

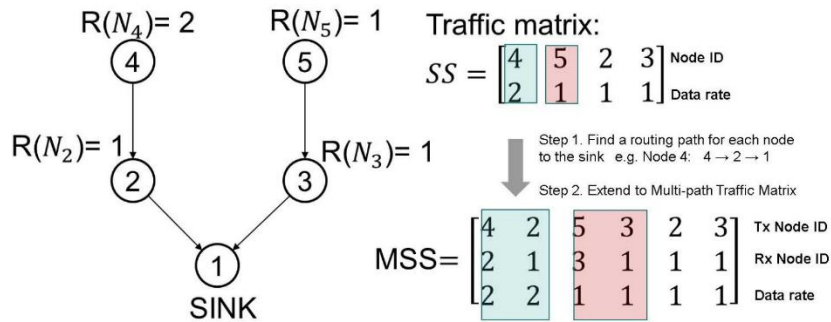


Non tutti gli schedule sono **corretti**:

- **Collisioni**: se riceve da troppi nodi nello stesso slot (numero di mittenti superiore al numero di radio).
- **Interferenze**: due trasmissioni nello stesso canale.

Non tutti gli schedule corretti sono **buoni**:

(b) è corretto ma non buono perché il nodo 2 deve attendere il prossimo frame per inviare i dati ricevuti dal nodo 4 al nodo 1 (sink), c'è un delay maggiore rispetto al caso ideale (a) .



1. Il PCE ottiene informazioni sui link e sui nodi.
2. Calcola la Traffic matrix (SS).

Per ogni nodo N_i è specificato il rateo $R(N_i)$.

3. Calcola la Multipath traffic matrix (MSS).

Per ogni link L_{ij} nella rete ne specifica il rateo.

I link sono $4 \rightarrow 2$ e $2 \rightarrow 1$, infatti abbiamo due colonne. L'elemento sulla prima riga è il nodo 4, nella seconda riga è il nodo ricevitore 2 e nella terza riga il rateo del link $R(L_{42})$. Notare che $4 \rightarrow 2 \rightarrow 1$ e $2 \rightarrow 1$ sono considerati due link differenti.

4. Il PCE calcola lo **schedule regolare** e lo schedule delle celle inutilizzate per le ritrasmissioni.

		Timeslot									
Channel offset		1	2	3	4	5	6	7	8	9	10
	2 → 1	4 → 2	2 → 1	4 → 2	2 → 1	4 → 2	2 → 1	2 → 1	2 → 1	2 → 1	3 → 1
	5 → 3	3 → 1	5 → 3	3 → 1	5 → 3	3 → 1	5 → 3	5 → 3	5 → 3	5 → 3	4 → 2

A → B Cell schedule based on known traffic

A → B Proportional Tentative Cell allocation

Problema: nelle celle aggiuntive il ricevitore si sveglierà, anche se il trasmettitore non ha nulla da trasmettere.

Soluzione: *end of queue notification* → flag nel messaggio che indica al ricevitore di non svegliarsi al prossimo slot perchè non ha più nulla da ricevere.

6TOP Decentralized Scheduling

Comprende due fasi:

1. **Scheduling Function** (*Quanti?*): Calcola il numero di slot necessari per comunicare con un vicino.
2. **6P Negotiation** (*Quali?*): Negoziare gli slot con tale vicino.

La SF può essere definita in base alle necessità, altrimenti può essere utilizzata quella di default.

6P

Il mittente A specifica le sue celle disponibili mentre il ricevitore B le sceglie in base a quali sono le sue disponibili.

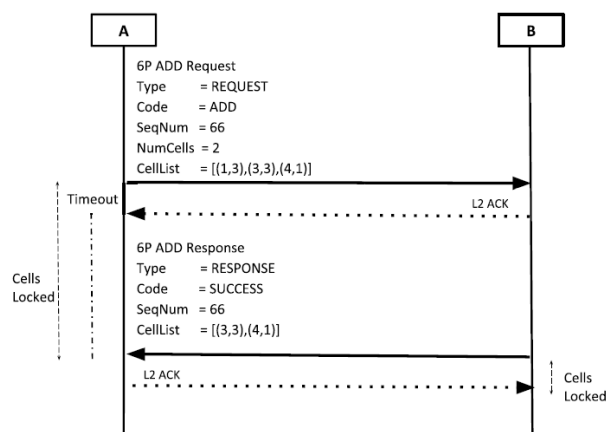
1. A invia un messaggio di richiesta ADD verso B.

Il nodo A specifica: il **numero di celle da allocare** e la **lista di celle che il nodo A ha attualmente libere** che possono essere utilizzate per la comunicazione.

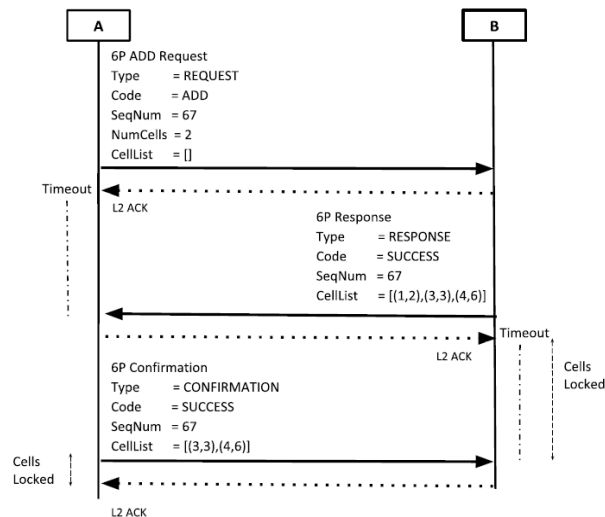
2. Dopo un pò B invia un RESPONSE, che include un codice.
3. Se SUCCESS, B ha accettato la richiesta di A.

Nella risposta B specifica la lista di celle attualmente libere (quelle mancanti rispetto alla ADD sono quelle scelte da B, ovvero quelle libere che erano libere anche per lui).

Cosa succede se B non ha slot liberi? manda un messaggio che non è di successo.



Variante (più lenta, poiché ha bisogno di un messaggio in più) dove è il mittente a scegliere le celle da usare mentre è il nodo B a indicare la lista di celle libere.



MSF – Minimum Scheduling Function

Calcola dinamicamente il numero di celle in base al traffico e alle condizioni della rete:

- NumCellsUsed > HIGH → aggiungere celle con 6P.
- NumCellsUsed < LOW → rimuovere celle con 6P.

MAX_NUMCELLS → ogni quante celle (di quelle assegnate) si deve fare il controllo.

NumCellsUsed → Numero di celle usate tra quelle negoziate in questo frame.

ALGORITHM 1: MSF Algorithm (*NumCellsElapsed*, *NumCellsUsed*)

Input:

NumCellsElapsed = Number of elapsed negotiated cells

MAX_NUMCELLS = Max number of elapsed negotiated cells

NumCellsUsed = Number of negotiated cells used for transmission

LIM_NUMCELLSUSED_HIGH = Threshold to add a new negotiated cell

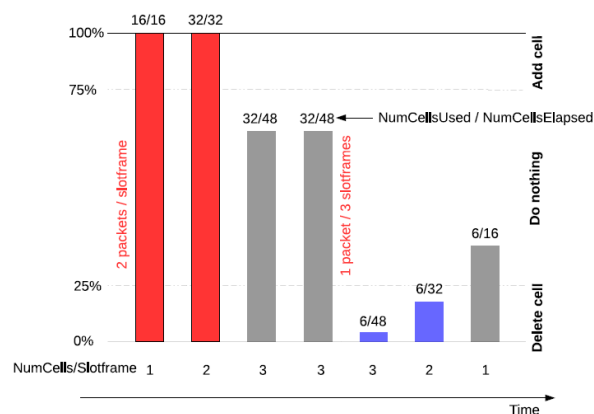
LIM_NUMCELLSUSED_LOW = Threshold to delete an unnecessary negotiated cell

Output:

ADD/DEL one negotiated cell

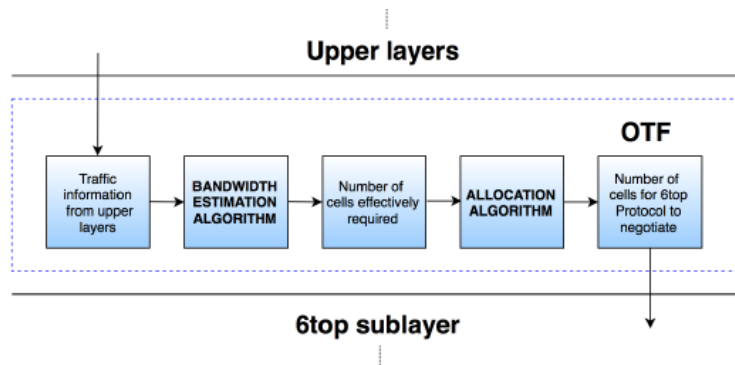
- 1 if *NumCellsElapsed* > *MAX_NUMCELLS* then
 - if *NumCellsUsed* > *LIM_NUMCELLSUSED_HIGH* then
 - trigger 6P to ADD one negotiated cell
 - else if *NumCellsUsed* < *LIM_NUMCELLSUSED_LOW* then
 - trigger 6P to DEL one negotiated cell

L'utilizzazione esprime quante celle la coppia di nodi usa di quelle che hanno predisposto. 100% indica che stano usando tutte le celle e quindi non ci sono celle di backup per affidabilità.



OTF – On-The-Fly Reservation

Stima la bandwidth per trasmettere il proprio traffico espresso in termini di numero di celle necessarie.



Formation process nelle reti TSCH.

Non c'è nelle slide. Processo di avvio di una rete TSCH: a partire da un nodo capofila (BR) fino alla **sincronizzazione e adesione** dei nodi.

1. Scoprire l'esistenza di una rete TSCH;

Il BR invia dei **Enhanced Beacon (EB)** in cui specifica le informazioni sul channel hopping: ASN, Channel Hopping Sequence, Slotframe ID e PAN ID.

2. Time Synchronization;

Quando un nodo riceve un EB, si sincronizza al clock del mittente (BR) e adotta lo stesso ASN.

3. Allinearsi al channel hopping schedule;

Esegue 6P (oppure riceve uno schedule dal PCE) per chiedere di allocare celle con i vicini.

4. Entrare nel DODAG e iniziare a comunicare.

Cosa è il frequency hopping?

Tecnica in cui si utilizza una frequenza diversa ad ogni trasmissione secondo una certa politica.

Multi-channel vs frequency hopping, perché due meccanismi?

Perché i loro scopi sono diversi, uno serve per aumentare le performance della rete, l'altro per aumentare l'affidabilità della comunicazione wireless.

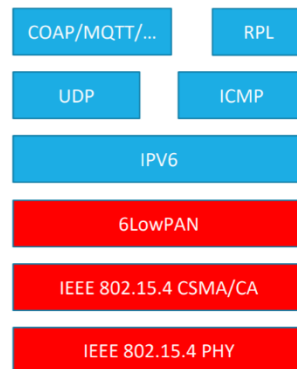
Quali sono gli effetti dei due?

Frequency hopping → Cambiare frequenza ad ogni trasmissione o ri-trasmissione aumenta l'affidabilità: riduce le interferenze e mitiga il multipath fading.

Multi-channel → Sfruttare molteplici frequenze aumenta il throughput aggregato e la capacità della rete poiché più nodi possono trasmettere contemporaneamente.

6LoWPAN

6LoWPAN in generale.



Livello di adattamento (sopra 6TOP) che permette l'**integrazione di IPv6 con reti TSCH**. Questo livello agisce su IPv6 per rendere le sue funzionalità più adatte a dispositivi limitati.

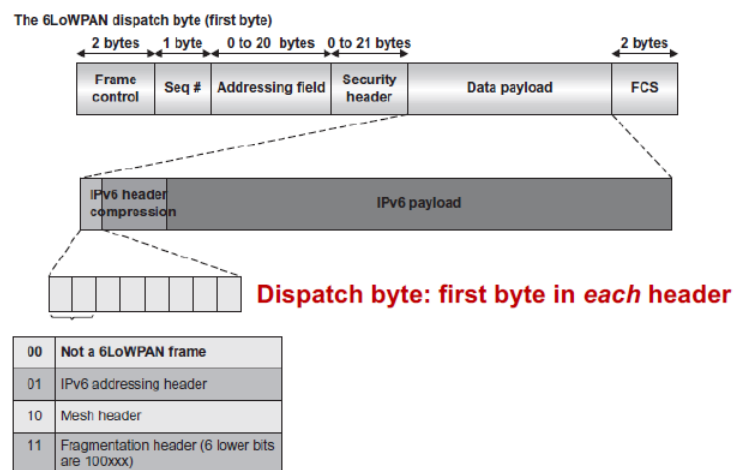
- **Compressione dell'intestazione:** riduce la dimensione dello header IPv6.
- Supporto per le due tipologie di indirizzo di 802.14.5:
 - **Lunghi** a 64 bit.
 - **Brevi** a 16 bit (quelli assegnati dal PAN coordinator).
- **Individuazione ottimizzata dei vicini:** adatta la ND ai dispositivi limitati.
- **Configurazione automatica della rete:** Configurare il proprio indirizzo IPv6 usando il prefisso del BR.
- Routing a livello data-link e a livello di rete:
 - **Mesh-Under:** a livello data-link. 6LoWPAN gestisce direttamente la trasmissione multi-hop.
 - **Route-Over:** a livello IP, ogni nodo agisce come un router IPv6 (es. tramite RPL).

Differenza tra una LowPAN e una LLN.

Una LLN è una rete generica composta da dispositivi limitati, mentre una LowPAN è una LLN costruita sopra 802.15.4

Compressione dell'header, differenza tra stateless e stateful compression

Il primo byte dell'intestazione IP compressa, il dispatch byte indica la tipologia di pacchetto.



Stateful compression

Stato condiviso tra i nodi → **considera trasmissioni precedenti**.

I nodi si accordano su `ContextID` → rappresenta un prefisso IPv6, che può essere rimosso dall'header.

Questa tecnica permette una grande diminuzione di byte di intestazione, ma non è adatta a LowPAN perché richiede negoziazioni periodiche e che si mantenga sempre uno stato in memoria.

Stateless compression

Non richiede informazioni condivise tra i nodi, si basa su regole predefinite (note a tutti) e valori impliciti, ad esempio:

- **Indirizzo sorgente/destinazione**: può essere derivato dal MAC.
- **Hop Limit**: si usa il valore standard.
- **Next Header**: si usa solo UDP.

IPHC - IPv6 Header Compression

Meccanismo definito per **compattare l'intestazione IPv6**: sfrutta la **ridondanza** e la **prevedibilità** delle LowPAN (sia elementi stateless che elementi stateful).

Alcune informazioni (come indirizzi o lunghezze) possono essere dedotte dal contesto della LowPAN. Il pacchetto IPv6 viene "ricodificato" in forma compressa, usando:

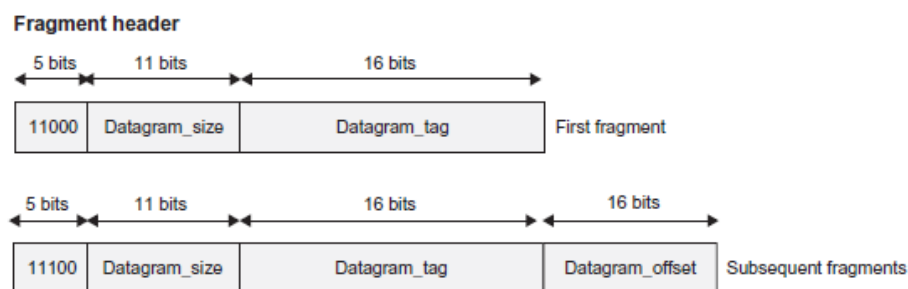
- **Flag binari** che indicano quali campi sono compressi, omessi o dedotti.
- **Valori abbreviati** o impliciti in base a regole note.

Il nodo ricevente, conoscendo le stesse regole di compressione e lo stesso contesto:

1. Legge i flag IPHC.
2. Ricostruisce l'header IPv6 completo.
3. Passa il pacchetto al livello IP → la compressione è **trasparente a IPv6**.

Frammentazione e campi dell'header frammentato

Fatta quando il pacchetto IPv6 supera i byte ammessi dal payload MAC.



Se vogliamo frammentare un pacchetto, dobbiamo considerare il giusto valore del dispatch byte:

- **11000xxx** → primo frammento.
- **11100xxx** → per tutti i frammenti successivi.

`Datagram_offset` → Per ordinare i frammenti.

`Datagram_size` → numero di byte del pacchetto, per capire quanti frammenti rimangono.

`Datagram_tag` → lega il frammento al pacchetto.

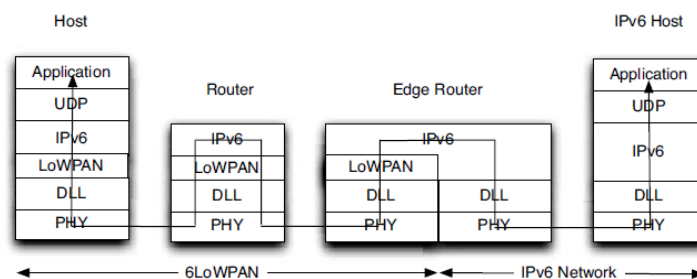
Riceve il 1° frammento → **riassembly timer** (in genere 60s), se non riceve tutti i frammenti entro il timer → abort.

Routing in 6LoWPAN.

Caratteristica	Route-Over	Mesh-Under
Livello di routing	Livello 3 (IP)	Livello 2 (Data link)
Chi legge l'intestazione IPv6	Ogni nodo	Solo il destinatario finale
Overhead	Maggiore (IPv6 completo ogni hop)	Minore (IPv6 letto una sola volta)
Protocolli di routing	RPL o altri IP	Proprietari o L2
Gestione frammenti	Ogni nodo ricompone e riframe	I frammenti viaggiano interi, end-to-end
Complessità nodi	Più alta	Più bassa

Route-over

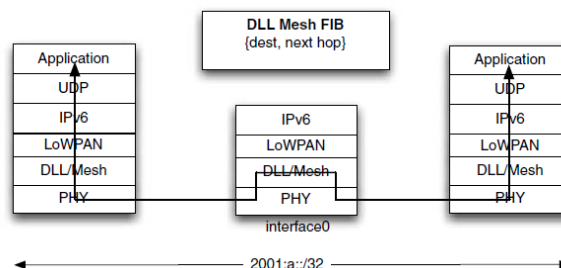
Ogni nodo si comporta da **router IPv6**: elabora l'intestazione IPv6 e prende decisioni tramite un algoritmo di routing. RPL fa parte di questi protocolli. Un nodo che riceve, legge l'intestazione IPv6 e inoltra secondo la propria tabella di routing. Il destinatario riceve ed elabora.



Route-over aiuta l'**interoperabilità della rete**, e per mantenere la raggiungibilità in un sottoinsieme tra i nodi.

Mesh-under

Routing a livello data-link, quindi tramite indirizzi MAC. Un nodo per inoltrare legge l'intestazione mesh e poi inoltra. Solo il destinatario elabora il pacchetto IPv6.



- **Più veloce:** Non si legge l'intestazione IPv6 ma solo quella data link.
- **Una rete mesh appare all'esterno come un'unica sottorete IP:** dal punto di vista del protocollo IP, tutti i nodi sembrano essere sullo stesso collegamento di rete.

Neighbor discovery in 6LoWPAN

Il Neighbor Discovery Protocol (NDP) serve a:

- **Scoprire gli altri nodi** sulla stessa rete (vicini).
- **Scoprire il router** e da lui ottenere il **prefisso link-local** da usare nella rete.

La procedura standard di IPv6 standard **non funziona bene per le LowPAN** perché:

- Ci sono molti domini di broadcast sovrapposti → il flooding ha effetti disastrosi.

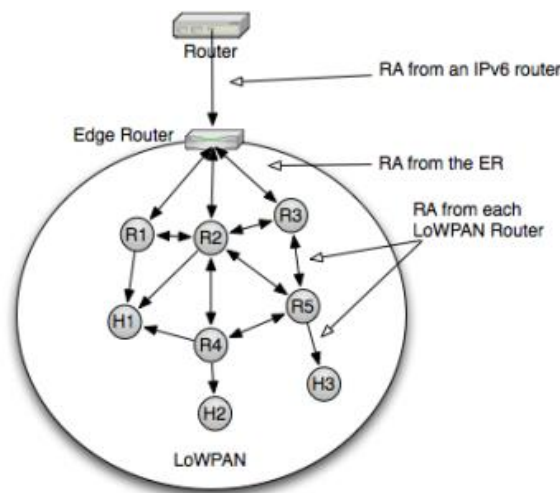
- Usa messaggi multicast → ogni nodo deve ascoltare sempre.
- Messaggi troppo grandi → Non stanno entro i limiti di 802.15.4 → frammentazione → overhead.
- I dispositivi spesso non sono sempre attivi, non contemplata da NDP classico.

6LoWPAN introduce una sua versione di NDP con semplici ottimizzazioni riguardo:

- **Scoperta dei vicini.**
- **Meccanismo di indirizzamento (Prefix Dissemination):** assegnare un indirizzo IPv6 ai dispositivi.
- **DAD:** tutti gli indirizzi (link-local) all'interno della LowPAN devono essere univoci.
- Introduce il ruolo del BR come entità centrale.

Prefix Dissemination

Procedura per rendere noto il prefisso link-local ai dispositivi IoT.



Le RA sono generate da un router IPv6 classico, contengono svariate informazioni (prefisso, ecc.).

1. Vengono inviate da una specifica interfaccia del router dove troveremo il BR.
2. Il BR invia periodicamente (o su richiesta) lo RA nei link della rete.

Tramite l'inoltro dei RA, ogni nodo scopre anche i suoi vicini.

3. Se ci sono router intermedi (6LR o R) lo propagano ad altri nodi vicini (H).
4. Nel RA c'è il prefisso, in autonomia genera il proprio indirizzo IPv6 (prefisso + interfaceID).
5. DAD.

Un nodo che entra nella rete, può scoprire il router locale inviando dei **Router Solicitation (RS)** in multicast, ai quali il router risponderà con un RA.

DAD – Duplicate Address Detection

In IPv6 ogni nodo verifica che il proprio indirizzo sia unico con un Neighbor Solicitation (NS) (mettendo come target l'indirizzo scelto) in modalità multicast. Se riceve una risposta vuol dire che quell'indirizzo è già in uso.

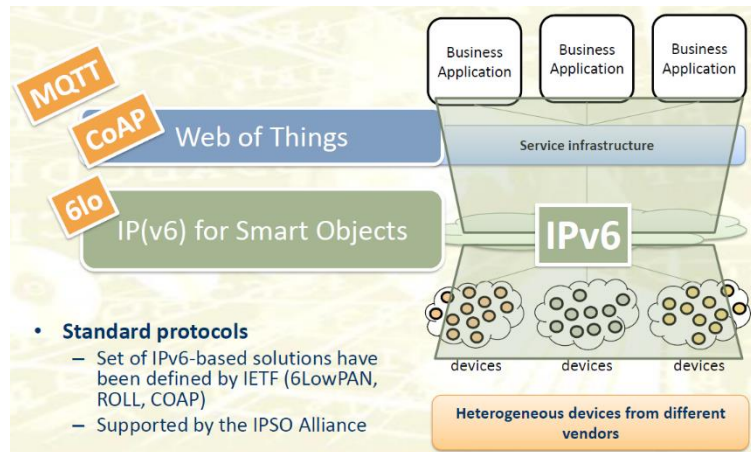
Questa procedura non va bene (richiede il multicast e che i nodi siano sempre in ascolto) in una LowPAN → sostituita da una **procedura centralizzata eseguita dal BR.**

1. **Prefix Dissemination.**
2. **Generazione:** Genera un indirizzo IPv6 (con il prefisso annunciato dal BR).
3. **Registrazione:** Invia un **NS unicast al BR**, con l'indirizzo IPv6 scelto e il proprio MAC.
4. **Controllo:** BR controlla se quell'indirizzo IPv6 è già registrato:
 - a. **Sì** → risponde con Neighbor Advertisement (NA) con status = duplicate
 - b. **No** → lo registra e risponde con NA con status = success

I binding (IPv6 ↔ MAC) all'interno del BR sono di tipo soft-state → scadono se non vengono rinnovati.

IoT Integration

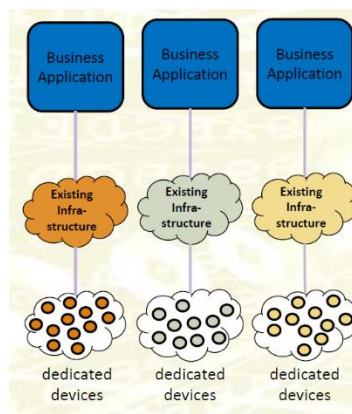
Come si integrano i sensori e gli attuatori con le soluzioni di cloud computing?



Sistemi verticali

Ha l'obiettivo di perseguire un **singolo scopo**, operando in **isolamento** rispetto ad altri sistemi verticali.

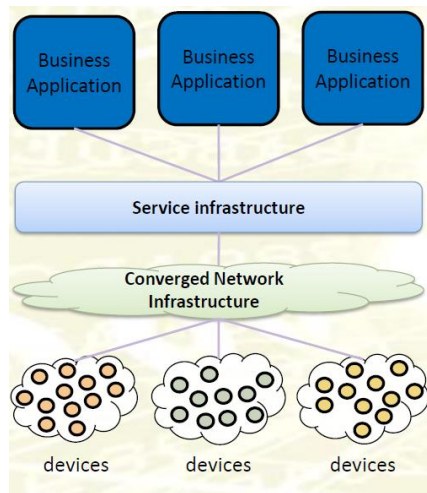
Ogni componente progettato per un singolo scopo → ogni costruttore crea il proprio **ambiente** per i propri dispositivi.



Sistemi orizzontali

Il nostro obiettivo.

Dispositivi diversi possano essere installati indipendentemente dal costruttore e senza essere legati ad una specifica applicazione. I dispositivi sono *“open”* e possono essere utilizzati da un'applicazione tramite un'interfaccia standard (service infrastructure).



Questa soluzione **riducono i tempi di sviluppo** → Molti componenti già esistenti e non devono essere realizzati da 0.

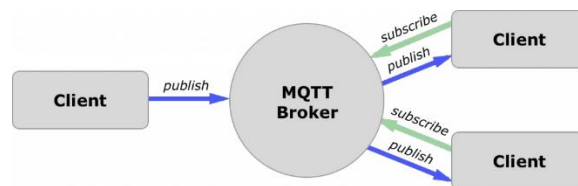
Molti cloud providers offrono **supporti IoT nativi**: package con moduli e scheletri per sviluppare applicazioni IoT.

MQTT

Che cos'è MQTT?

Protocollo di messaggistica di tipo **publish-subscribe**.

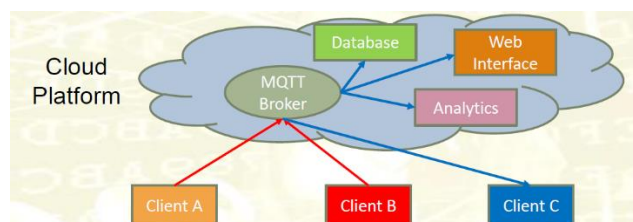
I client producono/consumano dati mentre il server (MQTT Broker) ha la responsabilità di ricevere e inoltrare dati da un client all'altro.



Ogni volta che un nuovo messaggio è **pubblicato**, il broker lo riceve, prende la lista di clienti **interessati** a ricevere quel tipo di messaggio (topic) e invia tale messaggio a tutti quei clienti. Non esiste una comunicazione di tipo client-client.

1. Problemi di MQTT

Il broker e l'applicazione sul cloud mentre i sensori nella WSN. I sensori inoltrano al broker i dati raccolti, mentre le applicazioni che si comportano come subscriber per ricevere i dati.



Pro:

- **Sviluppo semplice e veloce** usando i packages forniti dalla piattaforma cloud.
- **Non richiede l'installazione di una nuova infrastruttura**, l'infrastruttura e broker sono forniti dalla piattaforma cloud.
- **Scalabile con l'infrastruttura Cloud** (vantaggio principale).

Contro:

- **Cloud sempre coinvolto** il broker è nel cloud → interazione dispositivo-to-dispositivo non possibile.

- **Elevata latenza:** il cloud può essere fisicamente molto lontano.
- **Connettività persistente:** se broker irraggiungibile, il sistema non funziona, anche se due dispositivi potrebbero comunicare.
- **Sicurezza:** i dati escono sempre dalla WSN.

Web of Things

WoT in generale.

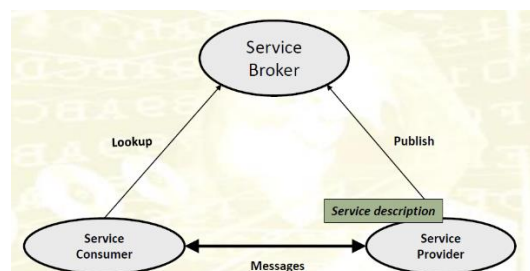
Il centro del sistema passa dal broker ai dispositivi: **l'applicazione contatta direttamente il dispositivo** (indirizzo IPv6) e chiede informazioni tramite un'interfaccia esposta dal dispositivo stesso.

- **Semplice.**
- **Energy Efficient:** Trasferimento di dati solo se necessario.
- **Interfaccia comune.**
- **Web of** → Stesse tecnologie e idee del Web.

Come si arriva al WoT da SOA?

SOA

Invece che organizzare un'applicazione in modo monolitico la si **divide in servizi indipendenti**. L'architettura SOA è composta da tre entità:



1. **Publish:** Il produttore pubblicizza il suo servizio al broker.

Invia con SOAP la descrizione del proprio servizio, realizzata con WSDL.

2. **Lookup:** Il consumer contatta il broker per ottenere la descrizione del servizio.

Una volta scaricate, il consumatore può contattare il produttore.

3. Il consumatore contatta direttamente il produttore tramite SOAP (protocollo per scambiare messaggi XML).

Non è adatta al mondo dell'IoT a causa della complessità dei messaggi e dei componenti.

- **Complessità eccessiva:** XML, WSDL e SOAP sono pesanti e verbosi.
- **Overhead elevato:** Grande volume di messaggi scambiati.
- **Modello sincrono:** SOA è sincrono (richiesta/risposta), mentre l'IoT può richiedere anche modelli asincroni e/o event-driven.
- **Difficoltà di scalabilità dinamica:** pensata per ambienti stabili, i dispositivi IoT si collegano/disconnettono frequentemente.

REST

Il mondo si è spostato da uno scenario user-to-machine, ad uno machine-to-machine (server-to-server), quindi serve un'interazione più diretta e con meno messaggi.

Le tecnologie del web vanno bene per una interazione machine-to-machine in un contesto cloud (senza dispositivi IoT)?

- **URI** va bene.
- **HTTP** va bene anche per portare dati generici.
- **HTML** non va bene → serve un linguaggio come XML per codificare ogni tipo di dato.

Le tecnologie del web vanno bene per una interazione machine-to-machine in un contesto IoT?

- **URI:** Va bene anche nel campo IoT.
- **HTTP:** non va bene per l'IoT poiché presenta un **ampio overhead e complessità**.
- **XML** troppo verboso e ridondante.

WoT

Quindi per i dispositivi IoT si è dovuto creare un nuovo protocollo di comunicazione che sostituisca http (CoAP) e nuove tecnologie di data encoding.

Quali sono le principali caratteristiche del WoT per cui si sposa bene con l'architettura a doppio layer in cui l'applicazione può “muoversi” tra il fog e il cloud?

La motivazione principale è che la comunicazione non passa attraverso un broker MQTT o altri intermediari.

L'applicazione, tramite un indirizzo IPv6 (unico al mondo) **contatta direttamente il dispositivo** come se fosse un server Web ad un'interfaccia usando un **protocollo semplice** come http o CoAP. Il dispositivo risponde a tale richiesta usando il medesimo protocollo.

Il fatto che l'applicazione sia nel cloud o nel fog non cambia nulla dal punto di vista del dispositivo.

Cos'è un approccio RESTful? Che origini ha? HTTP, SOAP, WSDL che vantaggi hanno?

REST è un approccio molto semplice: i server espongono delle risorse codificate con degli URI. I client effettuano delle richieste tramite http usando i metodi tradizionali GET, POST, ecc. Il server riceve le richieste alle suddette risorse e risponde al client sfruttando http.

SOAP e WSDL (in generale l'architettura SOA) permette di rendere un'applicazione molto **più mantenibile e organizzata** che altrimenti sarebbe sviluppata in maniera monolitica.

- SOAP è un linguaggio di comunicazione basata su scambio di messaggi XML.
- WSDL è un linguaggio per definire delle interfacce per dei servizi.

Qual è il vantaggio di usare HTTP al posto di SOAP+WSDL per dispositivi limitati.

HTTP risulta **più semplice** con **meno scambi di messaggi**. Tuttavia, risulta comunque inefficiente comparato a CoAP.

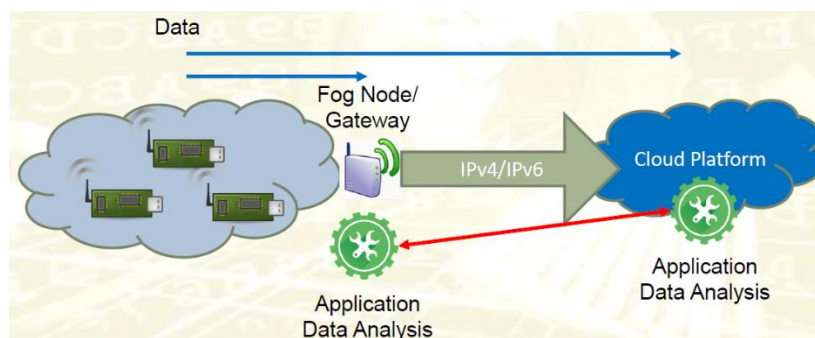
Fog Computing

Alcuni dei problemi di MQTT (elevata latenza, connettività con il cloud persistente) verrebbero risolti, se venisse spostata la logica verso dispositivi.

Il fog computing consiste nel portare l'**elaborazione e l'archiviazione** più vicino ai dispositivi IoT.

Una parte della logica applicativa sarà in esecuzione vicino ai dispositivi IoT (**bassa latenza e grande disponibilità**). Mentre l'altra parte della logica si trova nel cloud, per operazioni più pesanti ma meno urgenti.

Il risultato è un'**architettura a due livelli**: nel primo ci sono le risorse nel cloud, mentre nel secondo quelle in prossimità dei dispositivi.



MQTT non è adatto al fog computing, perché non sappiamo dove posizionare il broker.

Dovremmo metterne uno in locale e uno nel cloud? In tal caso, cosa succederebbe se esistesse un sensore di interesse per due applicazioni contemporaneamente ma una che gira in locale e una che gira nel cloud?

Se il gateway fosse l'unico broker MQTT diventerebbe un bottleneck. Ma se ne inseriamo due (uno nel cloud e l'altro nel gateway) non sapremmo come gestire i dati (ricordiamo che MQTT è un sistema di publish-subscribe).

MQTT non è adatto ad un'architettura multi layer (livello locale e livello cloud), dobbiamo quindi cambiare protocollo di comunicazione.

Data Encoding

Data encoding e SenML. Differenza con XML? Qual è la struttura generale di SenML? (nominare la semantica), struttura generale di un documento.

XML

Un documento XML ha un'architettura gerarchica, i dati sono inclusi nei documenti utilizzando un linguaggio di markup. Un documento XML per essere utilizzato deve rispettare due qualità:

- **Well-formed:** Il documento rispetta le regole di XML e quindi è interpretabile da una macchina.
- **Sensato:** Il contenuto deve avere un senso.

Lo schema XML è un meta-documento che specifica un set di regole che descrive che tipi di attributi deve avere il documento.

La struttura di XML è praticamente identica a quella di un documento HTML.

JSON

Linguaggio **molto meno verboso e ridondante di XML** dove ogni cosa è definita in maniera "chiave"- "valore".

Binary Encoding

Tecniche per ottimizzare linguaggi come XML e JSON. Entrambi sono **text-based**, quindi le macchine passano moltissimo tempo a fare il parsing.

Con il binary encoding, le informazioni sono codificate in maniera binaria, quindi i due standard EXI e COBOR (rispettivamente per XML e JSON). In questo modo il dispositivo può interpretare il documento senza fare parsing.

EXI e COBOR sono due standard che permettono di "tradurre" un documento XML e JSON in maniera binaria.

SenML

Non è un linguaggio di data encoding ma uno standard per la telemetria per JSON. Definisce come i dati devono essere interpretati (semantica) appunto per la telemetria.

- **Base Name (bn)** opzionale e indica il nome associato alle misure effettuate.
- **Base Time (bt)** opzionale e indica il riferimento temporale per ogni misura nell'oggetto

Semantica → cosa rappresentano i dati?

Questo arricchimento semantico consente a chi riceve i dati di **interpretarli correttamente**, favorendo **interoperabilità** tra dispositivi diversi.

```

{ "e": [
  { "v": 20.0, "t": 0 },
  { "sv": "E 24' 30.621", "u": "lon", "t": 0 },
  { "sv": "N 60' 7.965", "u": "lat", "t": 0 },
  { "v": 20.3, "t": 60 },
  { "sv": "E 24' 30.622", "u": "lon", "t": 60 },
  { "sv": "N 60' 7.965", "u": "lat", "t": 60 },
  { "v": 20.7, "t": 120 },
  { "sv": "E 24' 30.623", "u": "lon", "t": 120 },
  { "sv": "N 60' 7.966", "u": "lat", "t": 120 },
  { "v": 98.0, "u": "%EL", "t": 150 },
  { "v": 21.2, "t": 180 },
  { "sv": "E 24' 30.628", "u": "lon", "t": 180 },
  { "sv": "N 60' 7.967", "u": "lat", "t": 180 } ],
  "bn": "http://[2001:db8::1]",
  "bt": 1320067464,
  "bu": "%RH"
}

```

Name (optional if Base Name is present)	Name of the sensor or parameter. The Name attribute concatenated to the Base Name attribute must result in a <u>globally unique identifier</u> for the resource (an URI is recommended).
Units (Optional)	Units for a measurement value
Value (Optional if a Sum value is present)	Value of the entry. Only three basic data types: - Floating point numbers ("v" field for "Value") - Booleans ("bv" for "Boolean Value") and - Strings ("sv" for "String Value")
Sum (optional if a Value is present)	Integrated sum of the values over time
Time (optional)	Base Time + Time = Time when value was recorded (if zero, the time is roughly "now")
Update Time (optional)	A time in seconds that represents the maximum time before this sensor will provide an updated reading for a measurement

Perché non si usa HTML?

HTML è stato progettato per codificare delle pagine web → estremamente verboso.

Perché si introduce il data encoding?

Per organizzare i dati nel payload in maniera tale che un qualsiasi ricevitore possa essere in grado di leggere il contenuto (interpretarli è un altro discorso).

Cos'è EXI rispetto a XML?

EXI è una tecnica di binary encoding per documenti XML, permette di tradurli in stringhe binarie e fare in modo che il ricevitore perda molto meno tempo nel parsing.

Tre situazioni diverse, quale useresti dei tre (XML, JSON e SenML)?

- **XML** → Preferibile quando i messaggi da inviare sono altamente strutturati e in cui è necessaria verificarne la struttura.
- **JSON** → Preferibile in caso di messaggi poco strutturati e/o brevi senza unità di misura o altro.

- **SenML** → Preferibile (imposto) quando inviamo dati di telemetria che includono unità di misura e istante di campionamento.

CoAP – Constrained Application Protocol

Cos'è e quali sono le sue principali caratteristiche?

Alternativa ad HTTP da usare nei contesti IoT. Il paradigma è quello descritto dal WoT:

- I dispositivi si comportano come server ed espongono le risorse verso i client, identificandole con un URI.
- I client (applicazioni) usano queste risorse utilizzando CoAP.

Progettato per consentire **interazioni machine-to-machine efficienti**. Si assume che l'applicazione e i dispositivi possano comunicare direttamente e con connettività IPv6.

Differenze tra GET, POST, PUT, DELETE?

Riflettono le **CRUD** (Create, Read, Update, Delete) nel seguente ordine (POST, GET, PUT, DELETE).

- **GET**: recuperare dati. Per operazioni **sicure** e **idempotenti**.
- **POST**: inviare dati ad un server, che spesso risulta in un cambiamento nello stato interno della risorsa.
- **DELETE**: eliminare una risorsa.
- **PUT**: cambiare il valore di una risorsa.

Sicuro → non cambia lo stato interno del server.

Idempotente → chiamando un metodo molte volte in sequenza, il risultato restituito non cambia.

Differenze tra CoAP e HTTP?

CoAP **non è una versione “lightweight” di HTTP**, il concetto è lo stesso ma si basa su assunzioni diverse.

- **UDP** più leggero di TCP. UDP non implementa congestion control, ordinamento e affidabilità.
- **PARSING meno complesso**: HTTP è text-based, molto tempo nel parsing. Mentre CoAP è binary-based.
- **CoAP ha anche un modello ASINCRONO**: il client invia la richiesta e non aspetterà la risposta.
- CoAP ha capacità di:
 - **Proxy e caching** (come HTTP).
 - **Mapping CoAP-to-HTTP** (funzionalità che permette di tradurre un messaggio CoAP in un messaggio HTTP e viceversa).
 - **Comunicazioni multicast** (non in HTTP).
 - **Discovery**: consente ai client di trovare i server e scoprire le loro funzionalità (non in HTTP).

Come funziona il discovery in COAP?

Scoprire automaticamente dei dispositivi e capire quali sono le loro funzionalità. Composta da:

1. **Device discovery**: Trovare la lista di dispositivi.
2. **Resource discovery**: Trovare l'insieme delle risorse fornite dai dispositivi nella lista.

Device discovery

Operazione di tipo “link-local” ed è compiuta tramite **messaggi multicast di CoAP** e gli indirizzi multicast di tipo “All CoAP Nodes”, ovvero il messaggio verrà ricevuto da tutti i nodi che implementano CoAP.

Resource discovery

Recuperare la lista di risorse esposte da un server tramite **messaggi unicast di CoAP**.

Ogni server espone la risorsa: `"/.well-known/core"`, una lista che descrive ogni risorsa. Quando riceve una richiesta di resource discovery, il server invia il file.

Questa risorsa espone alcune funzionalità per rendere più efficace la resource discovery come il **query filtering**: **filtrare la lista di risorse** in modo da recuperare solo quelle di un certo tipo.

Modalità di proxy?

Il proxy riceve le richieste dai client e le inoltra all'origin server (dopo aver effettuato parsing e altre operazioni), quest'ultimo invia la risposta al proxy, il quale la inoltrerà al client.

Forward proxy

Un forward proxy non è trasparente ai client: **il client è conscio del proxy** → può fornire informazioni nella richiesta riguardo il proxy tramite due opzioni:

- `Proxy-uri`: quale proxy vuole usare.
- `Proxy-scheme`

Reverse proxy

Il proxy è installato nello edge della rete IoT e la nasconde dall'esterno.

Deve **scoprire i dispositivi ed esporre le loro risorse come se fossero sue**. Il client interagisce con il proxy senza sapere dell'esistenza della rete IoT. Migliora la sicurezza, il proxy può implementare molte funzionalità di sicurezza.

Quando è utile introdurre un proxy?

Il proxy spesso **non è un dispositivo limitato** ed è usato per aggiungere funzionalità al sistema:

- **Operazioni intermedie sui messaggi**:
 - **Parsing delle richieste** inviate dal client.
 - **Modificare le risposte** del server origin.
 - **Protocol translation (mapping http-CoAP)**
- **Sicurezza**: Può implementare procedure di sicurezza che sarebbero troppo pesanti per i dispositivi.
- **Caching**:
 - **Observing**: Se ci sono tanti osservatori, il server non deve tenere in memoria tante informazioni per notificarli tutti. Il server invia la notifica solo una volta al proxy e quest'ultimo notifica tutti.
 - **Caching classico**: Se un client effettua richieste alla stessa risorsa, il proxy può memorizzare la risposta per non interpellare il server.

Differenze tra MQTT e CoAP e perché è stato scelto uno rispetto all'altro? Quale consuma più energia tra i due?

Caratteristica	MQTT	CoAP
Modello	Publish/Subscribe	Request/Response (HTTP-like)
Trasporto	TCP	UDP
Peso (overhead)	Maggiore	Minore
Consumo energetico	Più alto (TCP)	Più basso (UDP)
Affidabilità	Alta (QoS)	Media (con conferme)
Sicurezza	TLS	DTLS
Adatto a	Cloud-centric	Fog/edge, dispositivi low-power
Supporto WoT	SI	SI (soprattutto via LwM2M)

MQTT è adatto ad un contesto più cloud-centrico in cui il broker e i client sono allo stesso livello. Non possiamo avere broker nel cloud e dispositivi nella WSN (stessi problemi detti prima).

- **CoAP è preferito quando si vuole risparmiare energia** e i dispositivi sono limitati. Particolarmente usato nel Fog/Edge.
- **MQTT** viene scelto quando serve una comunicazione **affidabile e scalabile**, tipicamente in scenari **cloud-centrici**, in cui la presenza di un broker e una rete stabile sono garantiti.

MQTT consuma più energia perché per una interazione M2M prevede l'utilizzo del broker mediante TCP.

Come avviene lo scambio di messaggi tra client e server? Quali sono i 4 tipi?

Quattro tipi di messaggi:

- **Confirmable (CON)**: richiedono ACK → affidabili.
- **Non-confirmable (NON)**: non richiedono ACK → non affidabili.
- **ACK**: risposta alla corretta ricezione di un CON.
- **Reset (RST)**: per resettare la comunicazione tra due endpoint.

CON e NON indicano il tipo del messaggio, il payload di questi messaggi può essere una richiesta o una risposta.

Ogni messaggio CoAP (CON e NON) è caratterizzato da due campi:

- **MessageID**: Legare il CON al suo relativo ACK.
- **Token**: Legare la richiesta alla risposta.

ACK e risposta non sono la stessa cosa: lo ACK è la conferma di avvenuta ricezione di un CON, mentre la risposta è il risultato di ciò che è contenuto all'interno della richiesta.

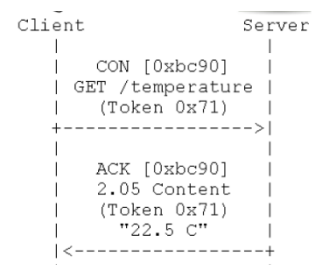
Comunicazione affidabile

Per certe applicazioni è necessario un trasferimento dati affidabile. CoAP sfrutta UDP → serve un meccanismo per gestire la perdita di pacchetti (prima risolto da TCP).

Introduciamo un **meccanismo opzionale di affidabilità lightweight**:

- **Rilevamento duplicati**: tramite un messageID nei messaggi.
- **Ritrasmissioni**: approccio stop&wait con backoff esponenziale.

1. Il client incapsula la richiesta in un CON.



Scegliendo il Con → il client ha deciso che la comunicazione deve essere affidabile.

2. Il server riceve la richiesta e:
 - a. **Risponde** con un ACK. Legato al CON grazie al messageID.
 - b. Può **rifiutarla** con un RST oppure **ignorarla**.

Ritrasmissione → Se non viene ricevuto ACK dopo un certo timeout → ritrasmissione.

Backoff esponenziale → Se il timer di un ACK scade → ritrasmissione con un ritardo randomico aumentato esponenzialmente. Questo fino al raggiungimento del numero massimo di ritrasmissioni → Abort.

Comunicazione inaffidabile

1. Il client incapsula la richiesta in un NON.
2. Il server riceve la richiesta e basta.

Risposta

- **Separated:** Alla ricezione di una richiesta (CON o NON) viene inviato un ACK vuoto. Quando è pronta viene inviata la risposta attraverso un messaggio CON o NON (dipendentemente dalla richiesta). La risposta avrà lo stesso token della richiesta ma diverso `messageID`.
 - Obbligatoria **per una comunicazione asincrona**.
- **Piggy-backed:** Incorporata dentro lo ACK.

Resource Observation

Polling

Richieste periodiche e continue alla stessa risorsa.

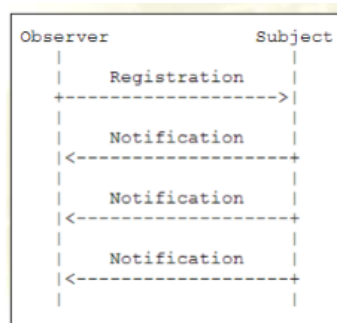
Inefficiente → il client deve svegliarsi, produrre una richiesta ed elaborare la risposta anche se il valore non è cambiato. Mentre il server deve fare il parsing della richiesta, produrre e inviare una risposta.

Observer

Il client riceve gli aggiornamenti direttamente dal server senza una richiesta esplicita.

Questo approccio è simile al publish-subscribe: un subscriber (client) si registra per ottenere messaggi di un certo topic (aggiornamenti di una certa risorsa) da un publisher (server origin o proxy).

Il sistema funziona tramite delle **unsolicited notification** → viene inviata una notifica ogni volta che l'aggiornamento della risorsa è pronto senza che il client invii una richiesta.



Una risorsa può essere osservabile oppure non osservabile (questo meccanismo non può essere utilizzato su di essa). Per interrompere il flusso basta che il client invii un RST.

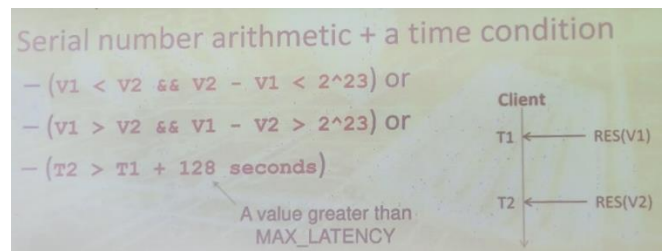
Importante → Le notifiche sono inviate al client dipendentemente da una policy del server (application depending).

Avere una notifica ogni volta che una risorsa cambia può portare ad una "tempesta" di notifiche → il server può implementare dei meccanismi per fare in modo che le notifiche vengono inviate solo in certe condizioni:

- **coap://server/temperature**
 - changes its state every second to the current reading of the temperature sensor
- **coap://server/temperature/felt**
 - changes its state to "cold" when the temperature reading drops below a certain pre-configured threshold, and to "warm" when the reading exceeds a second, slightly higher threshold
- **coap://server/temperature/critical?above=45**
 - changes its state based on the client-specified parameter value: every second to the current temperature reading if the temperature exceeds the threshold, or to "OK" when the reading drops below

Consistenza delle notifiche

Best effort → I messaggi potrebbero cambiare ordine durante il tragitto, il ricevitore deve utilizzare il campo `observe` presente nei messaggi di notifica per ricostruire l'ordine corretto.



Observing e Proxy

Il server deve memorizzare le informazioni di tutti i client che stanno osservando una certa risorsa. All'aumentare del numero di client le cose si complicano-

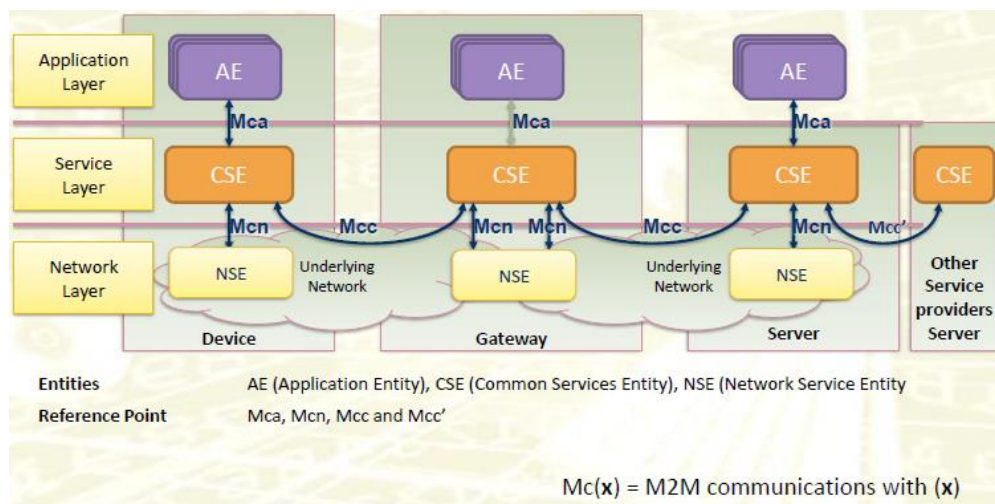
Reverse proxy → I client si registrano al proxy, ogni volta che una notifica viene ricevuta, la inoltra a tutti i client registrati.

Altro

OneM2M architettura generale, services (application e server, con le common service entity etc), servizio di discovery

Framework che supporta gli sviluppatori: permette di **ignorare dettagli implementativi dei dispositivi** → fare in modo che l'applicazione possa essere eseguita in ambienti e in hardware differenti.

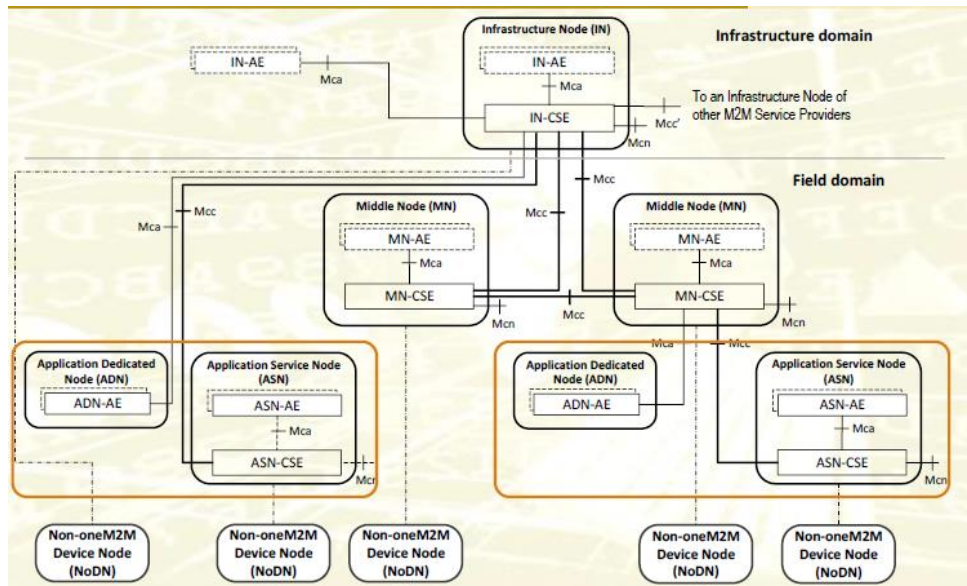
OneM2M è un **set di specifiche** standardizzate. Definisce i requisiti, l'architettura e le interfacce di una piattaforma IoT.



L'architettura funzionale indica la lista di funzionalità che ogni dispositivo deve implementare, gli sviluppatori vogliono avere delle funzionalità fornite dall'ambiente.

L'architettura di OneM2M è composta da **tre livelli**:

- **Network layer**: responsabile dell'implementazione di tutte le funzionalità di networking, è obbligatorio per ogni dispositivo altrimenti non potrebbero comunicare tra loro. Implementate nel modulo **NSE**.
- **Service layer**: le funzionalità base per ogni nodo. Implementate nel **CSE**.
- **Application layer**: le applicazioni, esse sono nominate **AE** (Application Entity). Possono essere eseguite nell'infrastructure node, nei middle node, oppure nei dispositivi IoT.



I dispositivi che implementano OneM2M sono di tre tipi:

- **Infrastructure domain:** Nodi che compongono il cloud, possiedono tutti e tre i moduli.
- **Field domain**
 - **Middle nodes:** Contengono NSE e il CSE (non tutte le funzionalità) e opzionalmente un AE.
 - **IoT Devices:** Contengono il NSE e il CSE.
 - **Application Dedicated Node:** Contiene il codice applicazione (AE).
 - **Application Service Node:** Possiede un AE per raccogliere dati dal dispositivo e il CSE.

CSE

Il **middleware** che gestisce le risorse, fornisce i servizi comuni e coordina la comunicazione tra AE e altre CSE.

- **Gestione risorse:** creazione, aggiornamento, eliminazione.
- **Registrazione:** Permette ad **AE** o **altri CSE** di registrarsi, annunciarsi e rendersi noti all'interno del sistema. Inoltre, permette di spedire informazioni riguardo un AE o un CSE ad un altro CSE in modo da usare i servizi M2M.
 1. Un AE invia una richiesta di **CREATE** al CSE con tipo di risorsa AE.
 2. Il CSE crea una risorsa **/AE**, che rappresenta l'app registrata.
 3. Se è un altro CSE, la risorsa è **/remoteCSE**.
- **Sicurezza:** Le funzionalità di autenticazione. Alcune applicazioni offrono funzionalità solo a chi ne ha il diritto.
- **Discovery:** Permette ad un AE o CSE per trovare risorse esistenti nel sistema fornite da dispositivi IoT o da altri CSE.
- **Subscription:** Permette ad un AE o ad un altro CSE di chiedere aggiornamenti riguardo una risorsa (in stile publish-subscribe).
- **Data management:**
 - Gestisce i dati generati dai dispositivi e li memorizza come risorse REST.
 - Deve permettere lo scambio di dati tra diversi AE.
 - Permette data storage e mediation, ovvero fornisce uno storage a breve termine: I dispositivi IoT possono sparire perché sono scarichi o altro; quindi, quando ritorneranno useranno questo servizio per fornire l'ultima informazione raccolta all'applicazione.

- Quindi le applicazioni usano queste memoria come intermediari tra loro e gli IoT.

- **Interworking:** interoperabilità con sistemi esterni (es. LwM2M, MQTT).

Sviluppato dal provider della piattaforma.

NSE

Permette al CSE di ignorare i dettagli riguardo al networking, concentrandosi solo sulle funzionalità.

- Identificazione del dispositivo.
- Gestione mobilità e localizzazione.
- Configurazione della rete, accesso.

AE

Rappresenta un'applicazione IoT. È il modulo che **crea, legge o gestisce dati** attraverso oneM2M.

- Registrarsi presso un CSE.
- Creare/gestire risorse (dati sensore, comandi, notifiche).
- Ricevere notifiche (via subscription).
- Interagire con altre AE (tramite il CSE come mediatore).

IPSO Alliance. IPSO smart object cos'è. con sistema dell'uri. in cosa aiuta lo sviluppatore. - che cos'è un URI

Aiuta lo sviluppatore **standardizzando il modo in cui vengono rappresentate e gestite le risorse**, semplificando lo sviluppo di applicazioni interoperabili. IPSO codifica le informazioni nella forma di tre numeri:

IPSO fornisce l'accesso ad un database pubblico, quale memorizza per ogni tipo di dispositivo, una grande quantità di informazioni.

- **ObjectID** identifica la famiglia/il tipo di dispositivo.
- **InstanceID** identifica sensori multipli dello stesso tipo che coesistono nella stessa WSN.
 - *Nel database non è specificato esplicitamente, e viene assunto come 0.*

Quindi **ObjectID/InstanceID** identifica uno specifico dispositivo in una WSN di tipo **ObjectID**.

- **ResourceID** è l'identificatore della risorsa nel dispositivo. Identifica la funzionalità che il sensore offre.

Quindi **ObjectID/InstanceID/ResourceID** identifica una specifica risorsa in uno specifico dispositivo in una WSN di tipo **ObjectID**.

Object	Object ID	Object URN	Multiple Instances?	Description
IPSO Light Control	3311	urn:oma:lwm2m:ext:3311	Yes	Light control object with on/off and optional dimming and energy monitor

Resource Name	Resource ID	Access Type	Mandatory	Type	Range or Enumeration	Units	Descriptions
On/Off	5850	R, W	Mandatory	Boolean			On/off control, 0=OFF, 1=ON
Dimmer	5851	R, W	Optional	Integer	0-100	%	Proportional control, integer value between 0 and 100 as a percentage.
Colour	5706	R,W	Optional	String		Defined by "Units" resource	A string representing a value in some color space
Units	5701	R	Optional	String			Measurement Units Definition e.g. "Cel" for Temperature in Celsius.
On Time	5852	R, W	Optional	Integer		s	The time in seconds that the light has been on. Writing a value of 0 resets the counter.

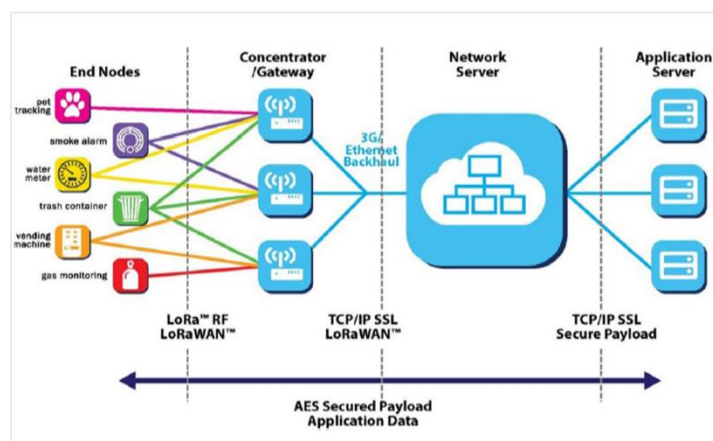
LowPower Wifi - LoRaWAN - Long Range WAN

Tecnologia di comunicazione wireless progettata per connettere **dispositivi a bassa potenza su lunghe distanze**. Fornisce una copertura di vari km a discapito di un rateo di trasmissione molto basso.

LoRaWAN Si tratta di un protocollo di comunicazione wireless pensato per reti IoT a basso consumo energetico, su lunghe distanze.

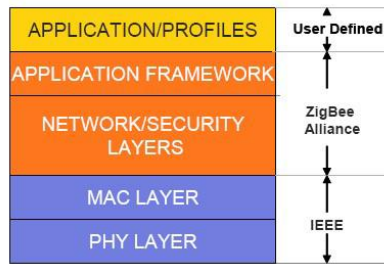
LoRaWAN è una tecnologia di tipo LPWA:

- **Grande range di trasmissione:** I gateway LoRaWAN possono trasmettere fino a 10 Km in aree rurali e fino a 3 Km in aree urbane (ci sono molti più ostacoli che riducono la distanza massima).
- **Basso bitrate:** tipicamente da 0.3 kbps a 50 kbps.
- **Ambienti urbani:** Queste reti riescono a coprire aree al coperto e edifici con più piani.
- **Sicurezza end-to-end:** AES a 128 bit.



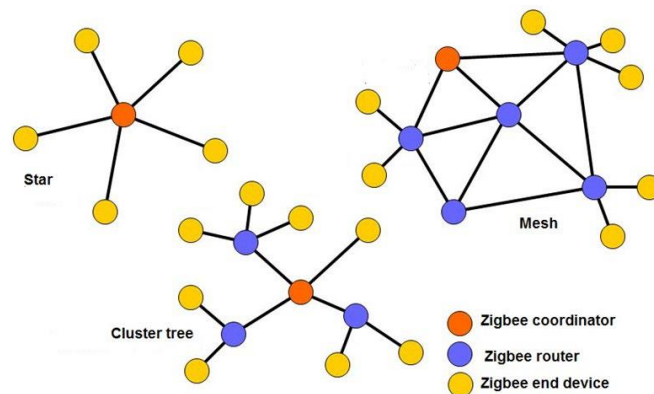
ZigBee

Se non vogliamo utilizzare IPv6 possiamo usare ZigBee.



Varie topologie. In ogni rete che utilizza ZigBee abbiamo al massimo 3 tipi di dispositivo:

- **ZigBee Coordinator [ZC]:** memorizza le informazioni e gestisce le attività nella rete.
- **ZigBee Router [ZR]:** nodi intermedi che inoltrano i dati verso la destinazione.
- **ZigBee End Device [ZED]:** sono gli end dispositivo e vengono considerati come nodi periferici. In genere raccolgono dati e li inoltrano ai Router.



AODV

Il protocollo di routing utilizzato nel livello di rete (Network Layer) dello stack ZigBee.